

Response to Reviewer Comments

We thank the (meta) reviewers for insightful comments and constructive suggestions. The comments and changes are highlighted in different colors (orange for Meta Reviewer, violet for Reviewer 1, red for Reviewer 2, and blue for Reviewer 3).

Meta Reviewer

1. The presentation requires further improvement, including the addition of more technical details (R#1.D1, R#1.D4), clearer visualizations (R#1.D2, R#3.W6), better explanations of key concepts (R#1.D3), a more thorough discussion of related work (R#1.D4), and correct problem formulation (R#3.W9).

Reply: Thank you for the comment regarding the presentation. We revise the manuscript accordingly.

(1) The addition of more technical details :

For R#1.D1, we revise “not much extra time” to a more precise quantitative statement in Section 1.3, Page 2.

For R#1.D4, we clarify the use of Proposition 1 in Section 3.1.2, Page 4, and add more discussion to better explain the argument.

(2) Clearer visualizations:

For R#1.D2, we clarify the dataset used in Figure 3 on Page 3.

For R#3.W6, we revise the experimental figures, including Figures 14 and 15, to provide clearer visualizations.

(3) Better explanations of key concepts:

For R#1.D3, we provide informal and intuitive explanations of Propositions 1-7 in Section 3.1, Page 4.

(4) A more thorough discussion of related work: For R#2.D5, we provide a more thorough discussion of related work in Section 5, Page 5.

(5) Correct problem formulation:

For R#3.W9, we correct the problem formulation in Section 3.1, Pages 3-4.

2. The significance and practical impact of the proposed algorithm should be more clearly highlighted (R#3.W1).

Reply: Following the suggestions in R#3.W1, we present how our proposal can work together with SIMD and SIMT on GPUs, and the corresponding trade-off evaluation in Section 4.8, Page 12.

3. the experimental evaluation is incomplete, lacking comparisons with additional baselines (R#1.D5, R#2.D3), results under different evaluation metrics (R#1.D6), more insightful analysis (R#1.D7-D8), comparisons with a broader range of related methods (R#2.D1-D2, R#3.W4), evidence of substantial performance improvements (R#2.D4, R#3.W2), validation on larger datasets (R#2.D6) and other dataset types (R#3.W3), as well as clearer descriptions of the experimental setup (R#3.W8).

Reply: We improve the experimental evaluation as follows.

(1) Comparisons with additional baselines:

For R#1.D5, we add the sorting-based method as a baseline for non-time-series datasets, and experiments about compression performance of this baseline in a new Section 4.7.1, Page 11.

For R#2.D3, we evaluate a heuristic that selects from a set of well-spaced candidate pack sizes (e.g., powers of two up to 2^{10}), and add this baseline and discussion in Section 4.7.2, Page 11.

(2) Results under different evaluation metrics:

For R#1.D6, we provide results under different evaluation metrics in Table 2, Page 8.

(3) More insightful analysis:

For R#1.D7, we add more insightful analysis about very small pack sizes optimal in many cases in Figure 2(b), in Section 4.2, Page 9.

For R#1.D8, we add a new set of experiments to analyze which data properties are predictive for the impact of our method, in Section 4.2, Page 9.

(4) Comparisons with a broader range of related methods:

For R#2.D1-D2, we add more comparisons with related methods, such as BitWeaving and Simple-8b in Section 4.4, Pages 10.

For R#3.W4, we add experiments to compare with Elf* in Section 4.4, Page 10. Moreover, our optimal pack-size selection is complementary to ALP and CuSZp2, and we use our method to further improve the compression ratios of both ALP and CuSZp2, in Section 4.3, Page 4.3.

(5) Evidence of substantial performance improvements:

For R#2.D4 and R#3.W2, we show the evidence of substantial performance improvements (15.5%) over datasets with small value bit-width, in Section 4.3, Page 4.3. Moreover, since savings of BP with optimal pack size can accumulate, we analyze the more significant improvement together with Sprintz.

(6) Validation on larger datasets:

For R#2.D6, we remove three small datasets (BP, SB, VC) with less than 10,000 points, and replace them with three larger integer datasets (CV, TF, TT) obtained from our industrial partner. The descriptions of these datasets are provided in Table 2 and Section 4.1.2, Page ??.

(7) Other dataset types:

For R#3.W3, we add the preprocessing explanation about floating-point datasets in Section 4.1.2, Page 9.

(8) Clearer descriptions of the experimental setup:

For R#3.W8, we add the description about the experimental platform and hardware configuration to Section 4.1, Page 8.

4. It is worth noting that the reviewers provided additional comments during the discussion phase, where authors need to pay specific attention to

Re R3#W3, it is a critical issue as floating-point values. Authors should address this carefully.

Re R3#W1, a clearer discussion of the hardware trade-offs, together with a stronger related work section, could address this to some extent.

Re competitors (baseline methods), authors are not required to compare against all alternative compression methods. However, it is necessary to at least make sufficient comparisons with related packing-based approaches.

Reply: We have carefully addressed all the points requiring specific attention in the revised version.

For R3#W3, we add the preprocessing explanation for floating-point values in Section 4.1.2, Page 9.

For Re R3#W1, we present below how our proposal can work together with SIMD and SIMT on GPUs, and the corresponding trade-off evaluation in Section 4.8, Page 12. And we also strengthen related work section by introducing FastLanes [2] and SIMDComp [15] in Section 5, Page 13.

[R#2.D5? R#1.D4 is not about related work]

for which point

Following the suggestions in R#3.W1, we summarize in one sentence

? it asks for float?

?

Reviewer #1

R1.W1. Please discuss reorder values and potentially add a new baseline (see below)

Reply: Thanks for your suggestions about a new baseline.

For R1.D5, we discuss value reordering (sorting) as a baseline for non-time-series datasets, and added corresponding experiments evaluating its compression performance in Section 4.7.1, Page 11.

R1.W2. The experimental analysis should have more depth (ideas below)

Reply: We analyze the experiments with more depth as follows.

For R1.D7, we provide an analysis explaining why very small pack sizes are often optimal in Figure 2(b).

For R1.D8, we analyze how data characteristics (e.g., smoothness and bit-length range) influence the effectiveness of our method, and build a model to predict the impact of optimal bit-pack lengths.

For R1.D9, we add error bars to Figures 14 and 15 to better illustrate variability.

R1.W3. Theorems lack intuitive explanations

Reply: We explain the intuition behind theorems as follows.

For R1.D3, we add an intuitive explanation for Proposition 1 (unimodality) in Section 3.1, Page 4. We also include similar informal explanations for the other two bounds in Propositions 6 and 7, to provide clearer intuition alongside the formal proofs in Section 3.2, Pages 5 and 5.

For R1.D4, we further clarify Proposition 1 to avoid potential misunderstanding in Section 3.1.2, Page 4.

R1.D1. Section 1.3 should give precise numbers instead of "not much extra time".

Reply: Thanks for the suggestion. We revise "not much extra time" to precise numbers "no more than 20% extra time" in Section 1.3, Page 2, based on the experimental results in Figure 14.

R1.D2. For Fig 2, the data set is unclear

Reply: We clarify that Figure 3 uses the PM10 dataset (listed in Table 2). The caption is revised to: "Total storage cost of 1024 values from the PM10 dataset (Table 2) under various pack sizes s ", in Page 3.

R1.D3. The new bound are interesting and formally proven, but the proofs should be accompanied with an informal explanation - something a reader can remember. For instance, the first conjecture (unimodality) holds because if there is a pack size X , then any pack size $k \cdot X$ will also include the packs of size X , thus there bit size cannot be smaller. That can also be visualized easily. Similar thoughts should be added for the other two bounds

Reply: In Section 3.1, Page 4, we add an intuitive explanation for Proposition 1 (unimodality). "Intuitively, if there is a pack size s , then any pack size $k \cdot s$ will also include the packs of size s . Hence, there bit size cannot be smaller and Proposition 1 holds".

For non-decreasing lower bound, the cost function for packing with pack size s will be no less than the cost of the first pack plus all subsequent values x_i using their own bit widths y_i . As s increases, the first pack becomes larger, and more values adopt larger bit widths, thus $L(s)$ is non-decreasing. For non-increasing lower bound, the cost function with pack size s will be no less than the cost of all values x_i using their own bit widths y_i . As s grows, the

number of packs decreases and the cost of bit widths decreases, thus $L'(s)$ is non-increasing.

We also include the informal explanations for the other two bounds in Propositions 6 and 7 to provide clearer intuition alongside the formal proofs in Section 3.2, Pages 5 and 5.

R1.D4. If I get proposition 1 right, the algorithm actually has to fully check only prime numbers (where no unimodality can hold). Please add this, if correct, to make the idea clearer

Reply: Thank you for the thoughtful observation. Proposition 1 implies that the algorithm actually has to fully check prime numbers (where no unimodality can hold). The unimodality property only arises among pack sizes with multiplicative relationships (i.e., composite numbers). Only such pack sizes can benefit from pruning based on unimodality. In Section 3.1.2, Page 4, we clarify this point to avoid potential misunderstanding.

R1.D5. While in time series the order of values is important, in other settings this is not the case. If order is not important, values could be sorted first, leading to values of similar length being close in the list. Such an input should be very well "packable" with a well-guessed pack size. I suggest adding this method as a baseline for non-TS data sets. Further, the idea of sorting values first (same complexity as the bounds) should be discussed somewhere (or maybe refuted)

Reply: Following the suggestion, we add the sorting-based method as a baseline for non-time-series datasets, and experiments about compression performance of this baseline in Figure 17. Furthermore, we discuss this idea including its complexity about bound in a new Section 4.7.1, Page 11.

We agree that, unlike time-series data, order might not be important for some non-time-series datasets. We test the compression performance with and without sorting for BP-All (i.e., traversing all possible pack sizes without pruning) and BP-Prune-RMQ (i.e., the method using pruning and RMQ to determine the pack size) on non-time-series datasets. If the values are sorted in descending order, values with similar bit width are together. As a result, sorting reduces the bit width needed for each value and makes them more packable. Hence, sorting improves the compression ratio of BP, as shown in Figure 17(a). Moreover, the maximum bit width of the current pack can be determined from the first value, and thus the pack size can be estimated more quickly. The methods with sorting have less compression time in Figure 17(b).

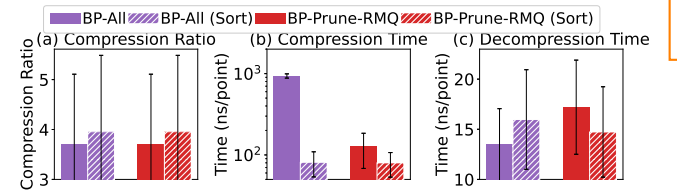


Figure 17. Impact of sorting non-time-series data on bit-packing with optimal pack size

R1.D6. Table 2 should show stddev and min/max of bit length

what is BP(All), >explain all the methods and exp settings in the reply

check font size in all figs!

why the relationship of decompression

no need to thank / agree in each point

also revise "without much extra time cost" in the conclusion

PM10 with 0 improvement in fig 12?

how visualize

there bit size?

give explanations for all three propositions in the reply, one by one

Reply: We add the stddev and min/max of bit length in Table 2, Page 8. When the stddev is large or the difference between the maximum and minimum bit lengths is significant, values with smaller bit widths within the same pack have to be stored with a larger bit width. Hence, the larger stddev and difference between the max and min bit width, the worse compression ratio is.

R1.D7. Why are very small pack sizes optimal in so many cases in Fig 11b? That's seems highly counterintuitive to me. Please analyze and explain

Reply: The key reason is the trade-off between the bit-width cost and the value cost. As shown in Figure 3 and suggested by Proposition 1, the bit-width cost decreases as the pack size s increases, with a relatively steep reduction when s is small. In contrast, the value cost typically increases more slowly with s . However, after applying Sprintz, the local variability of values is amplified, making larger packs more likely to include outliers with higher bit-widths. This effect offsets the benefit of larger s , leading to smaller pack sizes being optimal in many cases, as observed in Figure 2(b). We add this explanation to Section 4.2.

R1.D8. In Section 4.2, you should analyze whether there are properties of data series that are predictive for the impact of your method. For instance, I would expect a larger impact, the smoother the values change. What is the impact of the bit length range? Etc. Can we build a model which predicts the impact of optimal bit pack lengths?

Reply: In Section 4.2, we add a new set of experiments to analyze the properties predictive for the impact of our method.

1. We study the relationship between data smoothness and the relative compression improvement of our method comparing to fixed-pack-size method. We quantify smoothness as the mean of difference between adjacent bit widths, i.e., mean of bit width delta. A larger mean of delta represents adjacent bit widths differ more on average, indicating that the data fluctuates more. As shown in Figure 11(a), we observe that datasets with larger fluctuations (i.e., less smooth) tend to benefit more from our method.

2. We also analyze the impact of the bit-length range. By finding the optimal pack size, each value can save about 1 bit. However, this improvement becomes marginal when the bit length is large. As shown in Figure 11(b), a larger bit-length range generally correlates with smaller improvements.

3. We explore whether the impact can be predicted using a learned model such as random forest model. Using the above features, the model already achieves strong predictive performance ($R^2 = 0.953$, $\rho = 0.977$) in Figure 11(c). By incorporating 8 additional features, including mean, standard deviation, median, change count, 25th percentile, 75th percentile, run-length mean of bit widths, and proportion of zeros, the prediction accuracy is further improved.

R1.D9. What is the data underlying Fig. 13 and 14? If it is averaged over all data series, error bars are in order.

Reply: The data underlying Figures 14 and 15 is the average of all data series. As suggested, we add error bars to Figures 14 and 15. Even at the minimum and maximum bounds of these error bars, our algorithm for finding the optimal pack size still achieves a better compression ratio than algorithms using a specified pack size.

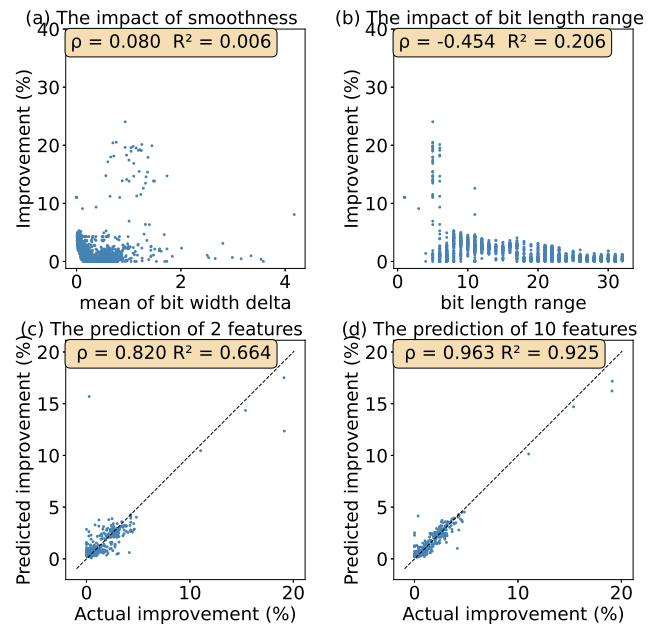


Figure 11: Data characters predict the impact of optimal pack sizes

Reviewer #2

R2.W1. Incomplete experimental comparison

Reply: Thank you for the suggestions to improve the completeness of the experimental comparison.

For R2.D1, we add experiments and discussion on BitWeaving and FastLanes in Section 4.4, Page 10 and Section 4.8, Page 12.

For R2.D2, we add comparison experiments and discussion on Simple-8b in Section 4.4, Page 10.

For R2.D3, we evaluate a heuristic that selects from a set of well-spaced candidate pack sizes (e.g., powers of two up to 2^{10}), and add this baseline and discussion in Section ??, Page 11.

R2.W2. No explanation of what dataset characteristics affect compression performance

Reply: We explain the dataset characteristics that affect compression performance, such as bit-width of values, in Section ??, Page 9, for R2.D4.

R2.W3. Narrow RW section

Reply: According to R2.D5, we expand the related work to better cover existing studies in this area. In particular, we now cite and briefly discuss representative methods such as Elf* [17] and Simple-8b [4] in Section 5, Page 12.

R2.D1. The paper only considers some baselines and does not compare against existing alternative compression methods like the ones mentioned in related work. Even though not bit-packing variations themselves, they are relevant.

Reply: As suggested, we include additional baselines from the related work, e.g., bit-packing variations BitWeaving and FastLanes.

BitWeaving introduces a bit-level columnar layout that enables efficient parallel bitwise operations and improves scan performance,

while achieving worse compression ratio than BP due to additional bits for alignment. Therefore, BP-Prune-RMQ achieves better compression ratio than BitWeaving as depicted in Figure 16, and we also add the discussion to Section 4.4, Page 10.

In addition, FastLanes mentioned in related work is a state-of-the-art library for optimizing bit-packing performance. We therefore extend our evaluation by integrating FastLanes as a backend to accelerate our method, and describe this optimization in Section 4.8, Page 12. As shown in Figure 19, FastLanes can further improve the compression time of our approach.

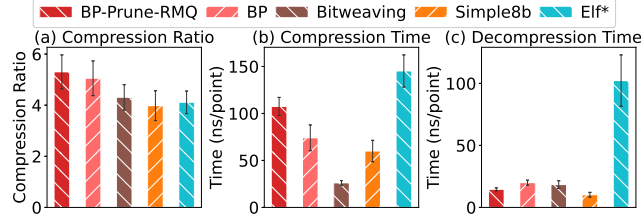


Figure 16. Comparison with other algorithms

R2.D2. Continuing from D1, even some very relevant bit-packing methods like Simple-8b are not considered.

Reply: In the revision, we include Simple-8b in our evaluation, and the results are reported in Figure 16, Section 4.4, Page 10.

Simple-8b uses a selector-based scheme that packs multiple integers into a 64-bit word with a uniform bit width. While efficiently decompressing, this design can be suboptimal if values within a group have different bit widths, leading to wasted space. In contrast, our method adapts the packing strategy with optimal pack size, resulting in better compression ratios. We also observe that our method achieves comparable compression and decompression time to Simple-8b in Figure 16.

R2.D3. Why not just try heuristically some values (even some pre-defined well-spaced ones) and pick the best? That would be faster than the proposed $O(n \log n)$ and in the end it might not affect compression rate much. This could be a useful baseline.

Reply: Thank for suggesting the interesting heuristic selection by using a small set of predefined pack sizes. In the revision, we evaluate this heuristic in Section 4.7.2, Page 11.

According to Figure 2, small pack sizes are often favored by BP. For the heuristic method, we only consider candidate pack sizes smaller than a threshold λ , and pick the best pack size from the range $[1, \lambda]$. We further conduct an experiment to study how compression performance varies with λ against our method BP-Prune-RMQ, which considers all n candidate pack sizes with pruning and RMQ. As shown in Figure 18, this approach reduces compression time compared to our method, but leads to a slight degradation in compression ratio. This baseline highlights the trade-off between compression ratio and time.

R2.D4. Compression improvements are shown but are at most modest. That's ok considering that bit-packing savings can accumulate for example when storing very large time series data. But still there's variability from dataset to dataset and it's not explained

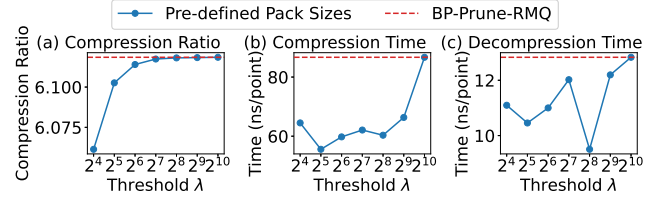


Figure 18: Compression performance of bit-packing with pre-defined pack sizes and all pack sizes

why certain datasets have better improvements while some get almost nothing.

Reply: It is true that the compression improvements can be modest and almost nothing in certain datasets. The reason is that this study does not propose a new compression method, but to determine a proper parameter of pack size for the vanilla Bit-packing encoding. It is thus not surprising that the improvement might not be significant, if the vanilla Bit-packing encoding already works well by setting a fixed pack size, e.g., dataset PM10 in Figure 12(a). As shown in Table 2, the dataset has large value bit-width and variance.

Nevertheless, for those datasets with small value bit-width, such as dataset TT with 6 bits, 1 bit gain per value could be considerable. The improvement is thus more significant, leading to 15.5% in Figure 12(a). For the same reason, the gain of Sprintz in Figure 12(b) is generally more significant than that of Bit-packing in Figure 12(a). As indicated in the review, bit-packing savings can accumulate, and the value width becomes smaller after Sprintz processing.

We add this discussion on variability from dataset to dataset in Section 4.3, Page 10.

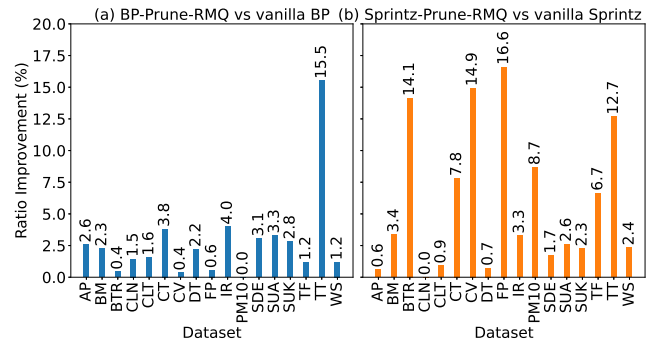


Figure 12: Improvement of compression ratio of BP-Prune-RMQ and Sprintz-Prune-RMQ

R2.D5. Related work is too short. A lot of work has been done in this field and some very relevant papers like the one that introduced Simple-8b are not cited.

Reply: Following the suggestions, we expand Section 5 to include additional relevant work on bit-packing encoding and related compression techniques. In particular, we now cite and briefly discuss Elf* [17] and Simple-8b [4] in Section 5, Page 5.

Directly, operating on the IEEE754 floating-point representation (i.e., sign, exponent and mantissa) is often inefficient, since even small fluctuations in values can lead to large differences in their binary representation. We first convert floating-point values into integers via scaling (i.e., multiplying by a fixed factor based on the maximum decimal precision), and then apply bit-packing to the integers.

Simple-8b [4] employs a selector-based scheme to pack multiple integers into a 64-bit word, achieving high decompression speed. However, Simple-8b can not change pack size to save the storage cost.

R2.D6. Several datasets are used which is good. But some are very small for their results to bear any meaning and it might even confuse the evaluation.

Reply: Referring to the suggestion, we remove three small datasets (BP, SB, VC) with less than 10,000 points, and replace them with three larger datasets (CV, TF, TT) obtained from our industrial partners. Cyber-Vehicle (CV) tracks series data pertaining to the operational status of concrete mixer trucks. TY-Fuel (TF) comprises data concerning the fuel consumption of various vehicle engines. TY-Transport (TT) encompasses data associated with the transportation duration of trucks. The descriptions of these datasets are provided in Table 2.

We rerun the experiments of Section 4 on the updated datasets, e.g., in Figure 12. The overall trends and conclusions remain consistent with the original results, indicating that our findings are robust across larger and more representative datasets.

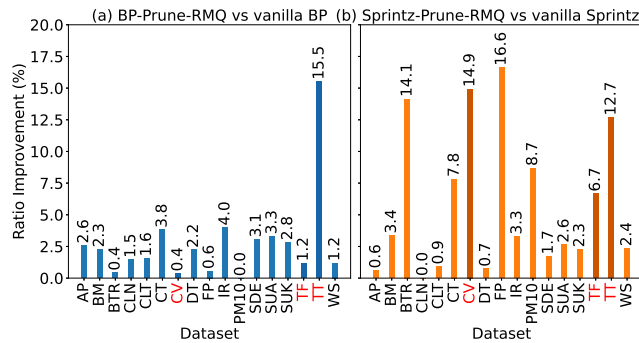


Figure 12: Improvement of compression ratio of BP-Prune-RMQ and Sprintz-Prune-RMQ

Reviewer #3

R3.W1. There is a fundamental conflict between the proposed algorithm and modern hardware trends. State-of-the-art libraries (e.g., FastLanes, SIMDComp) rely heavily on SIMD instructions, which typically require fixed and aligned block sizes for vectorization. The proposed method's approach of selecting arbitrary optimal pack sizes contradicts this requirement. The paper fails to discuss this critical trade-off, ignoring the fact that leveraging SIMD (or SIMT on GPUs) often yields order-of-magnitude improvements in processing speed.

Reply: Thank you for this important comment regarding modern hardware support and vectorization. We present below how our proposal can work together with SIMD and SIMT on GPUs, and the corresponding trade-off evaluation in Section 4.8, Page 12.

Note that the data are compressed in pages in practice. We determine a fixed pack size for each page, and thus is compatible with vectorization. Moreover, pack sizes aligned with multiples of 8 are preferred, e.g., 16 in FastLanes [2] or SIMDComp [15], so that they can be easily batched in 256 bits for vectorization. When the selected optimal pack sizes do not aligned with multiples of 8, such as 15, vectorization might not be fully utilized. Hence, there is a trade-off on compression ratio and time.

Figure 19 shows that with vectorization, i.e., SIMD, FastLanes and SIMT, the compression time can be significantly improved compared with the scalar implementation. The optimal pack sizes determined by our pruning and RMQ techniques indeed improve the compression ratio with the extra cost on time, i.e., the trade-off. Nevertheless, for the SIMT implementation, encoding uses per-thread writes with precomputed bit positions and `atomicOr`, which is particularly effective for non-aligned pack sizes (not multiples of 8). By selecting the optimal pack size, since the compressed data size reduces, the compression time is even improved a bit.

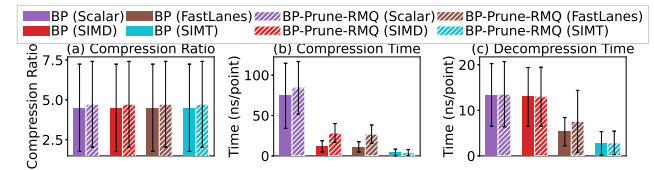


Figure 19: Compression performance of bit-packing optimal pack size with SIMDComp and Fastlane

R3.W2. The marginal gains of the proposed method are not significant. As observed in the experimental section, specifically Figure 12(a), the improvement in compression ratio is minimal, often saving less than 1 bit per value compared to the fixed-size scheme.

Reply: Note that this study does not propose a new compression method, but to determine a proper parameter of pack size for the vanilla Bit-packing encoding. It is thus not surprising that the improvement might not be significant, if the vanilla Bit-packing encoding already works well by setting a fixed pack size, as illustrated in Figure 12(a). However, selecting the optimal pack size for the vanilla Bit-packing encoding is costly, e.g., by testing all possible pack sizes. Our major contribution is thus to reduce the time cost of pack size determination, as presented in Figures 14.

Nevertheless, the 1 bit per value gain could be considerable, if the value bit-width is small. For example, for the new dataset TT, the difference between the maximum and minimum bit widths is 6 bits, saving 1 bit per value leads to a 15.5% improvement, in Figure 12(a). Moreover, as also indicated by R2.D4, bit-packing savings can accumulate. For example, our proposal can further improve Sprintz by optimizing the pack size, in Figure 12(b). The gain is generally more significant than that of Bit-packing, since the value width becomes smaller after Sprintz processing.

To better clarify the practical value of our solution, we add the discussion in Section 4.3, Page 10.

adjust text

where is Elf* [17] cited

what efficient, why, too complicated sentences, check all

double check

R2.D4, R3.W2

double check

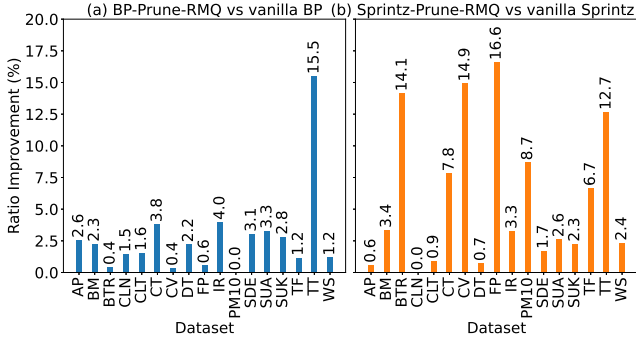


Figure 12: Improvement of compression ratio of BP-Prune-RMQ and Sprintz-Prune-RMQ

R3.W3. The experimental evaluation utilizes datasets commonly used for floating-point compression. However, the authors do not explain how bit-packing is applied to floating-point data. Given that the structural representation of floating-point numbers differs significantly from integers, a clear explanation of the preprocessing or mapping mechanism is required.

Reply: Note that the commonly used floating-point datasets are often with fixed decimal precision. Following the same line of preprocessing by scaling [18], we map the floating-point numbers to integers, i.e., multiplying by 10^κ where κ is the number of significant digits after the decimal point. The complex structural representation of floating-point numbers (sign/exponent/mantissa) can thus be compressed more easily by bit-packing as simple integers. We add the preprocessing explanation in Section 4.1.2, Page 9.

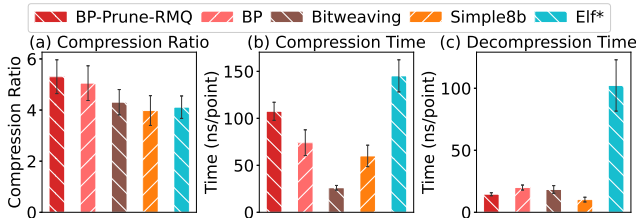


Figure 16. Comparison with other algorithms

R3.W4. The comparison with baselines is insufficient:

1.As noted in W3, since the target is floating-point data, the baselines are limited to Sprintz and fixed-width bit-packing. Modern floating-point compression algorithms (e.g., ALP[1], Elf*[2]) are missing. Furthermore, methods like ALP and CuSZp2[3] include optimized variants of bit-packing; the authors should include comparisons against these relevant approaches.

2.The experiments primarily compare against a fixed size of $s=8$, whereas Figure 13 indicates that a fixed width of $s=16$ often yields better results. The comparison should be made against the best-performing fixed configuration.

[1] Afrozeh A, Kuffo L X, Boncz P. Alp: Adaptive lossless floating-point compression[J]. Proceedings of the ACM on Management of Data, 2023, 1(4): 1-26.

[2] Li Z, Li R, Xu X, et al. Adaptive Encoding Strategies for Lossless Floating-Point Compression[J]. IEEE Internet of Things Journal, 2025.

[3] Huang Y, Di S, Li G, et al. cuSZp2: A GPU lossy compressor with extreme throughput and optimized compression ratio[C]//SC24: International Conference for High Performance Computing, Networking, Storage and Analysis. IEEE, 2024: 1-18.

Reply: As suggested, we add more baseline comparison.

1. For floating-point data, we compare with the modern floating-point compression algorithm Elf* [17], in Figure 16, Section 4.4, Page 10. Owing to the effective scaling preprocessing, as mentioned in R3.W3, the complex structural representation of floating-point numbers (sign/exponent/mantissa) can be compressed more easily by bit-packing as mapped integers. The corresponding compression ratio is thus better than Elf*.

For variants of bit-packing, note that we do not propose a new compression algorithm, but optimize the parameter selection of the existing bit-packing encoding. Hence, our pack size determination can also be applied to ALP [3] and CuSZp2 [11], i.e., complementing them. As shown in Figure 13, the pack sizes determined by our pruning and RMQ techniques further improve the compression ratios of both ALP and CuSZp2. The improvement on CuSZp2 is not as significant as ALP, since it is lossless compression with high ratio already. We add this discussion in Section 4.3, Page 10.

2. Following the suggestion, instead of a fixed size of $s = 8$, we update the experiments (e.g., in Figure 12) by using the best-performing fixed configuration, i.e., $s = 16$ for BP and $s = 8$ for Sprintz according to Figure 14, and revise the description in Section 4.3, Page 9.

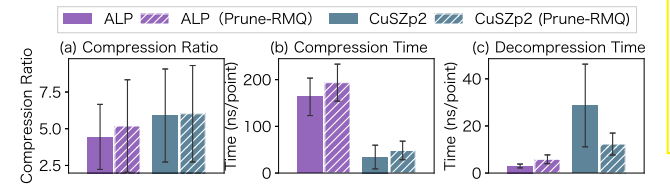


Figure 13: The compression performance of ALP and CuSZp2 with (without) optimal pack size

R3.W5. Some descriptions in the experimental section are unclear.

1. The calculation method for the "improvement" in Figure 12 and the "pruning rate" in Figures 14 and 15 needs to be explicitly defined.

2. Figures 13 and 14 do not clearly specify which dataset(s) were used to generate the results shown.

Reply: We add experimental descriptions as follows.

1. We add the following calculation method for the "improvement" in Figure 12 to Section 4.3, Page 9, and add the following definition of "pruning rate" in Figures 15 and 21 to Section 4.6, Page 11. The "improvement" is calculated as the compression ratio of BP (Prune-RMQ) or Sprintz (Prune-RMQ) minus that of BP or Sprintz, divided by that of BP or Sprintz. We define the pruning rate as the ratio of the number of pack size candidates pruned by our Propositions 1-5 to the total number of possible pack sizes n .

double check

revise "compared to using a fixed pack size 8 across all pages" in sect 4.3

R1.D9, R3.W5.2

2. For clarity, and unless stated otherwise, the results in Figures 14 and 15 (original Figures 13 and 14) present the average performance of each method across all datasets from Table 2, rather than the results from any single dataset. We add the statement about which dataset(s) were used to generate the results in Section 4.1.3, Page 9.

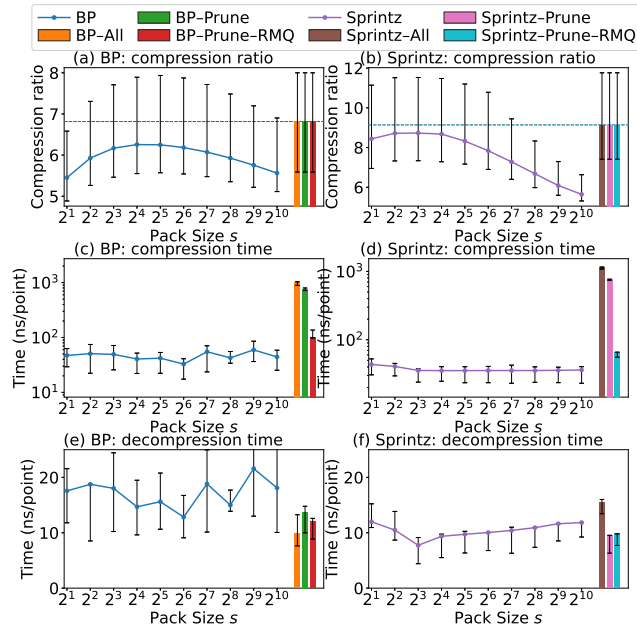


Figure 14: Performance under various fixed pack size s

R3.W6. The presentation of experimental figures needs improvement:

1. In Figures 13 and 14 (subplots e and f), the large scale of the y-axis makes it impossible to discern differences in decompression time.
2. The use of overlapping colored lines makes it difficult to distinguish between different schemes.
3. Regarding compression time, the focus should be on the comparison between the optimized BP and traditional fixed BP. Due to the large y-axis scale, the difference is obscured. For lightweight encoding schemes, even minor differences in compression speed are non-negligible and should be visualized clearly.

Reply: We improve the experimental figures as follows.

1. For subplots (e) and (f) in Figures 14 and 15 (Figures 13 and 14 in the previous version), we use a smaller scale of y-axis to discern differences in decompression time.
2. To avoid overlapping colored lines, we use bars instead, e.g., in Figure 14.
3. Regarding compression time, we use log scale in y-axis in the new Figures 14 and 15, in order to show the difference of the optimized BP and traditional fixed BP. As shown, our optimization with pruning and RMQ techniques takes only a bit more time than the traditional BP with fixed pack size. Obviously, the total cost of testing all these fixed pack sizes to find the optimal one is several times higher than our proposal.

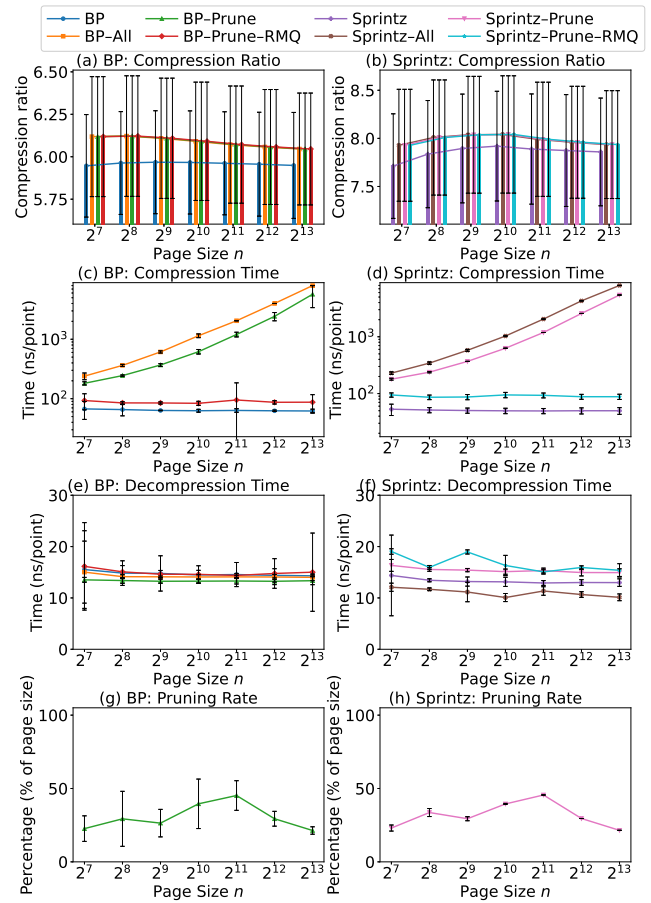


Figure 15: Performance under various page sizes n

R3.W7. Although the paper mentions system integration, the actual performance impact after deployment in the real system is not demonstrated.

Reply: Following the suggestion, we demonstrate the actual performance evaluation in the real system deployment of Apache Ts-File, in Section 4.9, Page 12.

In addition to the CPU cost of compression and decompression, the system integration incurs IO cost in writing and reading the compressed data. As shown in Figure 20(b), while determining the optimal pack size by our pruning and RMQ takes more CPU time, the corresponding IO cost is significantly reduced, due to better compression. The total write and read time costs of the system are indeed improved.

R3.W8. The experimental platform and hardware configuration specifications are missing.

Reply: We add the following specifications in Section 4.1, Page 8.

All experiments were conducted on a server equipped with two AMD EPYC 7713 CPUs (64 cores each), supporting the AVX2 instruction set, which is utilized by our SIMD implementation. The server is also equipped with five NVIDIA A100 GPUs (sm_80, Ampere architecture), which are used for the SIMT-based compression and decompression kernels.

Sprintz-optimized? does not know what Prune-RMQ means, similarly in other figs

not one dataset? then how to address the error bar concern

no improvement in (b) >the improvement is more significant on SprintZ in fig 12[The dataset with the highest compression ratio has not improved much. The mean of compression ratio is close to the compression ratio of this dataset, thus it seems

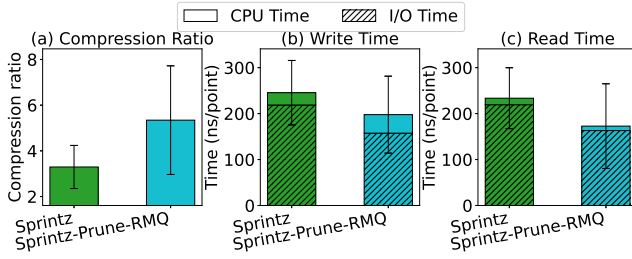


Figure 20: Compression performance impact after deployment in the real system

R3.W9. Most propositions in the paper rely on Proposition 1, but its formulation contains several errors, casting doubt on the reliability of the proofs:

1. The statement "and for all $s^* \leq s_1 < s_2$ in S , $C(s_1) > C(s_2)$ holds" is incorrect. For a downward unimodal (convex-like) function, the cost should increase after the minimum, so it should likely be $C(s_1) \leq C(s_2)$.
2. The symbol s'' is used without prior definition in the text.
3. In the equation $\Delta T(s) = T(s) - T(s') = \Delta_1 + \Delta_2 > 0$, the inequality direction seems questionable; should it be < 0 ?
4. The derivation often assumes n/s is an integer, which is not guaranteed in practice.

Reply: Thank you for carefully checking Proposition 1. We re-verify the formulation and proof, and correct the typos you pointed out.

1. We correct the monotonicity statement for the downward-unimodal cost function in Page 3. It should be $C(s_1) \leq C(s_2)$ rather than $C(s_1) > C(s_2)$.
2. We now define s'' explicitly as $s'' = \gamma^{\beta''}$ and $\beta'' = \beta' + 1$, i.e., $s'' = \gamma s'$, to avoid ambiguity, in Page 4.
3. We correct the sign in the derivation of $\Delta T(s) < 0$ in Page 4.
4. Note that the proof does not require $\frac{n}{s}$ to be an integer. Indeed, the proof uses $\lceil \frac{n}{s} \rceil$, such as 4, 5, 6, 7 or 8. We state this more clearly in Section 3.1, Page 4.

R3.W10. Writing and organization issues:

1. On Page 6, the text states "As shown in Figure 6(a), the blue line represents the lower bound $L(s)$..." This appears to be a typo and should likely refer to $L'(s)$, as $L(s)$ corresponds to the bound in Proposition 2.
2. Figure 11 (Distribution of Optimal Pack Sizes) provides important motivation for the work and should ideally be moved earlier in the paper.

Reply: Thank you for the comments regarding writing and organization issues. We revise the manuscript accordingly.

1. We correct the typo on Page 5, Section 3.2, by revising " $L(s)$ " to " $L'(s)$ " to be consistent with the definition of the bound in Proposition 2.
2. To better motivate the work, we move Figure 2 (Distribution of Optimal Pack Sizes, Figure 11 in the original version) and its description to an earlier position in Section 1.1, Page 1.

"When $\lceil \frac{n}{s} \rceil$ equals $\dots$?"

On Optimizing Pack Size for Bit-Packing

Jinzhao Xiao
Tsinghua University
xjc22@mails.tsinghua.edu.cn

Juncheng Hu
Jilin University
jchu@jlu.edu.cn

Shaoxu Song*
Tsinghua University
sxsong@tsinghua.edu.cn

Jianmin Wang
Tsinghua University
jimwang@tsinghua.edu.cn

Abstract

Bit-packing serves as a foundational operator widely used in diverse data encoding and compression schemes. It operates by partitioning a sequence of n values into fixed-size packs and representing all values within each pack using a uniform bit-width. We observe that using a pre-specified, fixed pack size does not always yield the optimal compression ratio across all datasets. To address this issue, the most straightforward approach is to exhaustively search through all possible pack sizes to identify the one that minimizes storage cost. However, the quadratic time complexity of this exhaustive search is unacceptable for bit-packing, a method whose primary advantage lies in its compression speed. In this paper, we propose a novel scheme that leverages pruning techniques and range minimum queries to identify the optimal pack size with a complexity of $O(n \log n)$. Experimental results on real-world datasets demonstrate that our method achieves a significantly higher compression ratio compared to using a fixed pack size, while maintaining comparable compression time.

PVLDB Reference Format:

Jinzhao Xiao, Shaoxu Song, Juncheng Hu, and Jianmin Wang. On Optimizing Pack Size for Bit-Packing. PVLDB, 19(1): XXX-XXX, 2026. doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://github.com/thssdb/encoding-pack-size>.

1 Introduction

Bit-packing (BP) [10] is a fundamental compression operator widely adopted in various data compression schemes [5, 14, 33]. It groups values into fixed-size packs, for example, four values per pack, as shown in Figures 1(a) and (b). Each pack is assigned a dedicated bit-width, determined by the maximum number of bits required to represent any value within the pack after stripping leading zeros. In the first pack of the example, the value 13 has the maximum bit-width of 4, corresponding to its binary representation $[1101]_2$. Consequently, all other values in the same pack can also be encoded using 4 bits. To enable proper decoding, the compressed data must

store both the binary content of each pack and its associated bit-width, thereby allowing distinct bit-widths across different packs to be correctly identified.

1.1 Intuition

Pack size, specifying the number of values in a pack, is the only parameter of bit-packing. It is worth noting that the setting of pack size should neither be too small or too large as illustrated below.

When the pack size is small, it is more likely to specify a proper bit width for the values in a pack. For instance, as illustrated in Figure 1(a), all the 4 values in the first pack need a bit width of 4, without wasting any bit for storage. However, there are more packs when grouping n values into smaller packs, each of which needs to store the corresponding pack bit width. For the 64 values in Figure 1(a), we have 16 packs with size 4, each consuming 3 bits to record its bit width. The total storage cost is thus 252 bits, i.e., $4 * (4 + 3 + \dots + 3) = 204$ for storing values and $16 * 3 = 48$ for recording bit widths.

On the other hand, if the pack size is large, the number of packs becomes smaller, with less cost of recording bit width for each pack. For example, in Figure 1(c), a pack size $s = 32$ leads to only 2 packs for 64 values. While the cost of recording pack bit widths is much reduced, only $2 * 3 = 6$ bits, the corresponding value storage cost is significantly increased, i.e., $32 * (5 + 3) = 256$ bits. The reason is that there are many small values assigned together with larger ones in the same pack. Recall that all the values in a pack share the same bit width as the maximum value. For instance, the 5th value 6 only needs 3 bits but takes 5, same as others in the first pack. The total storage cost of $256 + 6 = 262$ bits is still high.

In Figure 2, we present the distribution of optimal pack sizes in all pages after Bit-packing (BP) and Sprintz. We can observe that the optimal pack size is not a fixed size (e.g., 8) for all values. For instance, in the case of Bit-packing, most optimal pack size is smaller than 60 in Figure 2(a). The optimal pack size indeed varies significantly across all datasets. It verifies the motivation of this study, finding the optimal pack size for each specific data.

1.2 Challenge

Therefore, it is essential to find the optimal pack size s for bit-packing that can minimize the storage cost of values and bit widths. Note that the optimal pack sizes vary significantly across different datasets as shown in Figure 2 below. Indeed, when applying bit-packing in different compression algorithms, the optimal pack size could be various as well on the same data. Therefore, a pack size pre-set or determined on some sample data could be sub-optimal.

*Shaoxu Song (<https://sxsong.github.io/>) is the corresponding author.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, Vol. 19, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

reduce
the size
of such
huge figs,
fig 1-4,
6-9, 18

R3W10 2

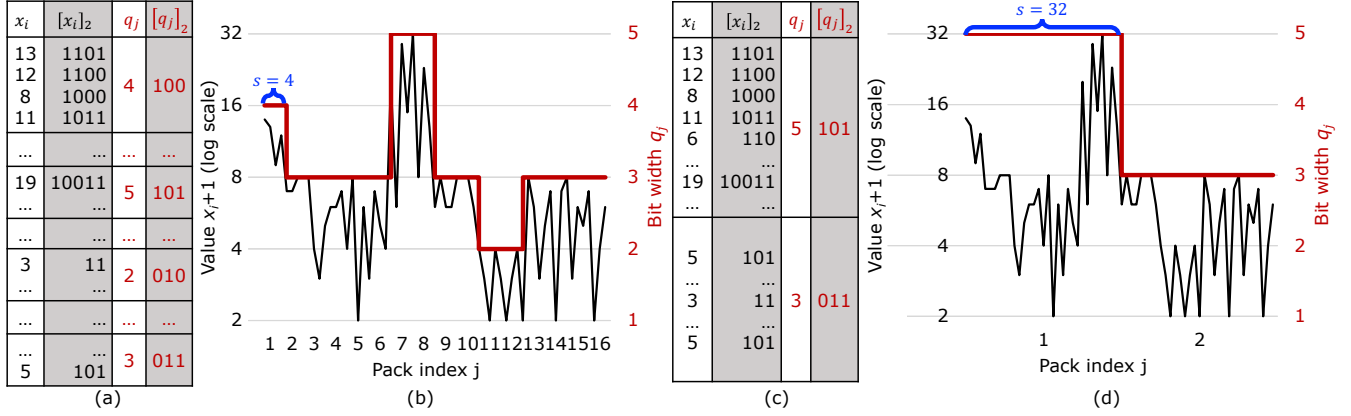


Figure 1: Bit-packing divides values into fixed-size packs, e.g., there are 4 values in a pack in the example of (a). The table shows values X , storing their binaries $[X]_2$, and bit widths Q of packs, storing the binaries $[Q]_2$. (b) visualizes values x_i and bit widths q_j of packs with size $s = 4$. (c) presents the packs of size $s = 32$, with the corresponding visualization in (d).

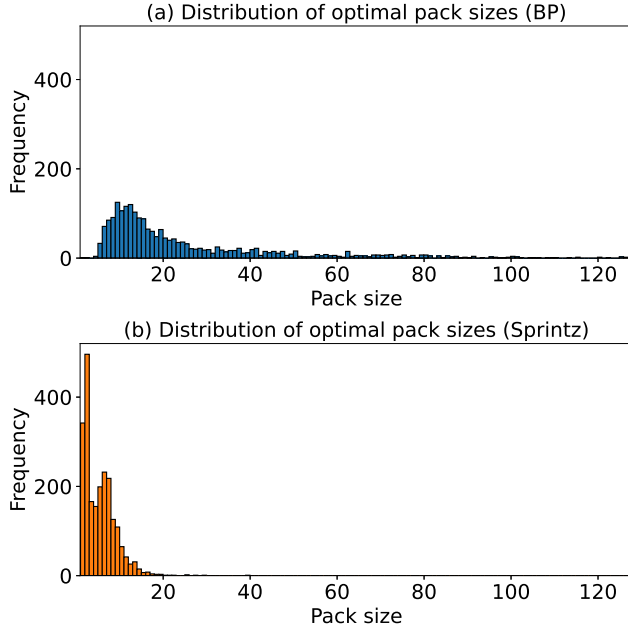


Figure 2: Distribution of optimal pack sizes

Given n specific values, a straightforward approach is to traverse from 1 to n as candidates of pack sizes s , and evaluate the storage cost of each candidate size s . However, it is unacceptable for bit-packing, which increases the time complexity to $O(n^2)$.

To address this problem more efficiently, we first investigate how the storage cost varies with the pack size. According to the aforementioned intuition, the pack size should not be too small or too large. For the two parts of storage cost, as shown in Figure 3 below, the bit width cost decreases with pack size s growing, but the value cost rises in a fluctuating manner. Consequently, the total storage cost is non-monotonic in general. In this sense, the key

challenge of the pack size optimization problem lies in identifying effective pruning opportunities to reduce the computational cost of evaluating various candidate pack sizes.

1.3 Contribution

In this paper, we propose an efficient algorithm for optimizing the pack size of bit-packing. Our method could significantly improve the compression ratio of bit-packing with comparable compression time. The major contributions are as follows.

- (1) We formalize the pack size optimization problem, and demonstrate how the storage cost of bit-packing depends on the pack size.
- (2) We propose an efficient solution for determining the optimal pack size, based on candidate pack size pruning and range minimum queries. While the storage cost does not strictly increase or decrease with the pack size, Proposition 1 leverages the modality of the storage cost function on subsets of candidate pack sizes and thus enables pruning. Moreover, we identify two lower bounds of the storage cost that are non-increasing or non-decreasing with the increase of pack size, in Propositions 6 and 7, another opportunity of pruning. Finally, we show the monotonicity of the cost function in the special cases of two packs, in Propositions 4 and 5, again possibly pruning half of the candidate pack sizes.
- (3) As an improvement of the fundamental operator bit-packing, we can cooperate with and enhance any existing compression methods that use bit-packing, such as Sprintz [5]. We have implemented our optimized bit-packing operator in Apache TsFile [32], and the code is available in [26].
- (4) We conduct extensive evaluation on real-world datasets. Our pack size optimization achieves up to 10% compression ratio improvement of bit-packing without incurring no more than 20% extra time. More than half of the candidate pack sizes can be pruned, demonstrating the effectiveness of our pruning strategies.

2 Problem Statement

We formally define the bit width of a value, the bit width of a pack, the storage cost, and the pack size optimization problem in the

Table 1: Notations.

Notation	Description
X, n	a set of values and the number of values
x_i, y_i	i -th value and its bit width
s, m	the size and number of packs
p_i, q_i	i -th pack and its bit width
z	bit width of the maximum bit width
$C(s)$	the storage cost of values with pack size s
$L(s), L'(s)$	the non-decreasing and non-increasing lower bound

section. Table 1 summarizes the common notations used in this work.

2.1 Preliminary

Given n values $(x_1, x_2, \dots, x_n)$, we define the bit width of each value x_i as y_i ,

$$y_i = \lceil \log(x_i + 1) \rceil.$$

Bit-packing encoding groups $X = (x_1, x_2, \dots, x_n)$ into m packs with s values, i.e.,

$$m = \lceil \frac{n}{s} \rceil.$$

Each value of the i -th pack is stored with the bit width q_i of the maximum value, i.e.,

$$q_j = \max_{(j-1)s < i \leq js} y_i,$$

forming a sequence of bit widths $Q = (q_1, q_2, \dots, q_m)$ that specifies the bit needed for each respective pack p_j with size s .

2.2 Pack Size Optimization Problem

The data compressed by bit-packing comprises two parts, the values and the bit widths. And the storage cost of values is the sum of these two parts.

DEFINITION 1 (STORAGE COST). The storage cost $C(s)$ of X with pack size s is

$$C(s) = s \sum_{j=1}^{m-1} q_j + (n - (m-1)s)q_m + m \lceil \log(\max_{1 \leq i \leq n} y_i + 1) \rceil, \quad (1)$$

where $(n - (m-1)s)q_m$ is the cost of the last possibly non-full pack.

According to Definition 1, we could define the pack size optimization problem as follows.

PROBLEM 1 (PACK SIZE OPTIMIZATION PROBLEM). Given X of n values, the pack size optimization problem is to find the optimal pack size s with the minimum storage cost $C(s)$ for X ,

$$\arg \min_{1 \leq s \leq n} C(s).$$

EXAMPLE 1. As shown in the data from Figure 3, when the pack size yielding the minimum storage cost is 13, the corresponding bit-width cost is 316 bits, the value cost is 4249 bits, and the total storage cost is 4565 bits.

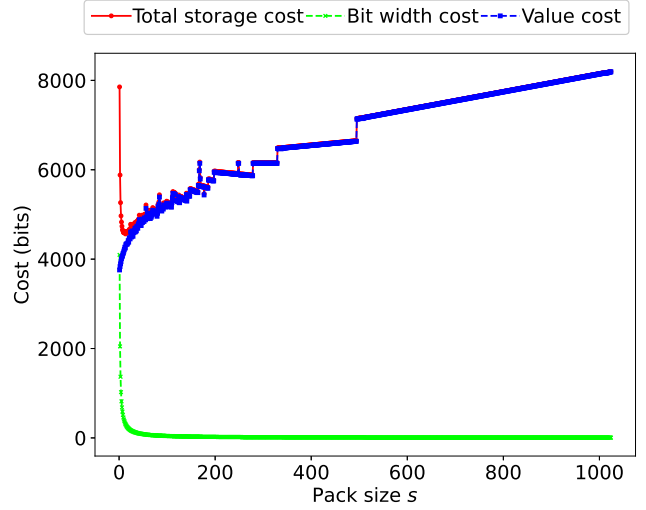


Figure 3: Total storage cost of 1024 values of dataset PM10 (Table 2) under various pack sizes s . It consists of two parts, the bit width cost and the value cost.

3 Pack Size Determination

If we directly enumerate each possible s , there are n possibilities in total, and computing the cost for each possibility requires $O(n)$ time complexity. The overall time complexity of the algorithm would be $O(n^2)$, which is excessively high and unacceptable for compression algorithms. Therefore, we present an efficient algorithm for determining the optimal pack size s that minimizes the storage cost function $C(s)$.

The key idea is to prune pack size candidates with unimodality, lower bounds and monotonicity in Sections 3.1-3.3. Then, the algorithm preprocesses the data to enable fast queries of the maximum bit width within any interval as depicted in Section 3.4. Finally, we propose the whole algorithm to determine optimal pack size in Section 3.5.

3.1 Unimodality for the Cost Function

According to Figure 3, the cost of bit widths is strictly decreasing with pack size s growing, but the cost of values is not strictly increasing. While there is a trend w.r.t. s as illustrated in Figure 3, the global unimodality of the cost function does not hold strictly on the entire set s of possible candidate pack sizes. Instead, we identify subsets of s , where unimodality is true in Proposition 1.

3.1.1 Unimodality on Subsets of Pack Sizes. Intuitively, when one pack size s_1 is a multiple of another pack size s_2 , there is a certain relationship between the costs corresponding to these two pack sizes. It is hoped that this relationship can be used to reduce the cost calculation for some pack sizes.

PROPOSITION 1 (UNIMODALITY FOR THE COST FUNCTION). Let $S = \{\alpha\gamma^\beta \mid \alpha\gamma^\beta \leq n, \beta \in \mathbb{N}\}$ be a subset of possible pack sizes, where $\alpha \geq 1$ and $\gamma \geq 2$ are integers, and $\mathbb{N} = \{0, 1, 2, \dots\}$ are natural numbers. The cost function $C(s)$ is downward unimodal, i.e., $\exists s^* \in S$, for all $s_1 < s_2 \leq s^*$ in S , $C(s_1) \geq C(s_2)$ holds, and for all $s^* \leq s_1 < s_2$

in S , $C(s_1) \leq C(s_2)$ holds.

PROOF. Let $T(s) = s \sum_{j=1}^{m-1} q_j + (n - (m-1)s)q_m$ be the cost of values and $B(s) = m \lceil \log(\max_{1 \leq i \leq n} y_i + 1) \rceil$ be the cost of bit widths. Assume that $s = \alpha\gamma^\beta$, $s' = \alpha\gamma^{\beta'}$, $s'' = \alpha\gamma^{\beta''}$ and $\beta'' = \beta' + 1 = \beta + 2$ hold.

(1) Firstly, we prove that $T(s)$ is no larger than $T(s')$. Let $Q = \{q_1, q_2, \dots, q_m\}$ and $Q' = \{q'_1, q'_2, \dots, q'_{m'}\}$ be bit widths of packs p_j and $p'_{j'}$. Since $s'' = \gamma s'$, we could get

$$\gamma q'_j = \gamma \max_{\gamma j \leq k \leq \gamma j + \gamma - 1} \{q_k\} \geq \sum_{k=\gamma j}^{\gamma j + \gamma - 1} q_k.$$

Thus, for previous $m' - 1$ packs, we could obtain difference $\Delta_1(s)$ between cost of $\gamma(m' - 1)$ packs p_j and $m' - 1$ packs $p'_{j'}$,

$$\begin{aligned} \Delta_1(s) &= s \sum_{j=1}^{\gamma(m'-1)} q_j - s' \sum_{j=1}^{m'-1} q'_j \\ &= s \sum_{j=1}^{\gamma(m'-1)} q_j - \gamma s \sum_{j=1}^{m'-1} q'_j \\ &= s \sum_{j=1}^{m'-1} \left(\sum_{k=\gamma j}^{\gamma j + \gamma - 1} q_k - \gamma q'_j \right) \leq 0 \end{aligned}$$

And we could infer the difference Δ_2 between the cost of the remaining packs,

$$\begin{aligned} \Delta_2(s) &= s \sum_{j=\gamma(m'-1)+1}^{m-1} q_j + (n - (m-1)s)q_m \\ &\quad - (n - (m'-1)s')q'_{m'} \\ &\leq s \sum_{j=\gamma(m'-1)+1}^{m-1} q'_j + (n - (m-1)s')q'_{m'} \\ &\quad - (n - (m'-1)s')q'_{m'} \\ &\leq s(m - \gamma(m'-1)) + \gamma(m' - m)q'_{m'} \\ &\leq (m + \gamma - 1 - \gamma m)sq'_{m'} \\ &\leq (\gamma - 1)(1 - m)sq'_{m'} \leq 0. \end{aligned}$$

Therefore, we get the difference $\Delta T(s)$ between $T(s)$ and $T(s')$,

$$\Delta T(s) = T(s) - T(s') = \Delta_1 + \Delta_2 \leq 0.$$

(2) Furthermore, it is no hard to prove $B(s) \geq B(s') \geq B(s'')$. Then, we could infer that

$$\begin{aligned} \Delta^2 B(s) &= B(s) + B(s'') - 2B(s') \\ &= \left(\lceil \frac{n}{s} \rceil + \lceil \frac{n}{s''} \rceil - 2 \lceil \frac{n}{s'} \rceil \right) \lceil \log(\max_{1 \leq i \leq n} y_i + 1) \rceil \\ &\geq \left(\frac{n}{s} + \frac{n}{s''} - 2 \frac{n}{s'} - 2 \right) \lceil \log(\max_{1 \leq i \leq n} y_i + 1) \rceil \\ &\geq \left(\frac{n}{s\gamma^2} (\gamma - 1)^2 - 2 \right) \lceil \log(\max_{1 \leq i \leq n} y_i + 1) \rceil. \end{aligned}$$

(2a) If $\gamma \geq 3$ satisfies, we could infer that

$$\begin{aligned} \Delta^2 B(s) &\geq \left(\frac{n}{s\gamma^2} (\gamma - 1)^2 - 2 \right) \lceil \log(\max_{1 \leq i \leq n} y_i + 1) \rceil \\ &\geq (1 * (3 - 1)^2 - 2) \lceil \log(\max_{1 \leq i \leq n} y_i + 1) \rceil = 0 \end{aligned}$$

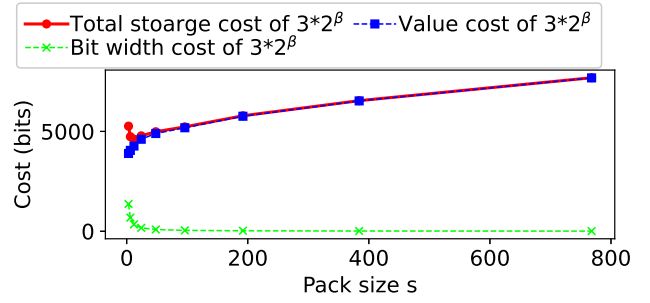


Figure 4: Example on unimodality of the cost function $C(s)$ on a subset of pack sizes $s = 3 * 2^\beta$, $\beta = 0, 1, 2, \dots$, according to Proposition 1

(2b) If $\gamma = 2$ and $\frac{n}{s''} = \frac{n}{s\gamma^2} \geq 2$, we could infer that

$$\begin{aligned} \Delta^2 B(s) &\geq \left(\frac{n}{s\gamma^2} (\gamma - 1)^2 - 2 \right) \lceil \log(\max_{1 \leq i \leq n} y_i + 1) \rceil \\ &\geq (2 * (2 - 1)^2 - 2) \lceil \log(\max_{1 \leq i \leq n} y_i + 1) \rceil = 0 \end{aligned}$$

(2c) If $\gamma = 2$ and $1 \leq \frac{n}{s''} = \frac{n}{s\gamma^2} \leq 2$, we could get $\frac{n}{s} \in [4, 8]$,

$$\begin{aligned} \Delta^2 B(s) &= \left(\lceil \frac{n}{s} \rceil + \lceil \frac{n}{s''} \rceil - 2 \lceil \frac{n}{s'} \rceil \right) \lceil \log(\max_{1 \leq i \leq n} y_i + 1) \rceil \\ &= \left(\lceil \frac{n}{s} \rceil + 2 - 2 \lceil \frac{n}{2s} \rceil \right) \lceil \log(\max_{1 \leq i \leq n} y_i + 1) \rceil \end{aligned}$$

Note that the proof does not require $\frac{n}{s}$ to be an integer. Indeed, the proof uses $\lceil \frac{n}{s} \rceil$, such as 4, 5, 6, 7 or 8. When $\frac{n}{s}$ equals 4, 6, or 8, the second difference $\Delta^2 B(s)$ can be expressed as $2 \lceil \log(\max_{1 \leq i \leq n} y_i + 1) \rceil \geq 0$. If $\frac{n}{s}$ equals 5 or 7, it becomes $\lceil \log(\max_{1 \leq i \leq n} y_i + 1) \rceil \geq 0$. $B(s) - B(s') < B(s') - B(s'')$ holds and $B(s)$ is a convex decreasing function, i.e., $B(s)$ decreases at a decelerating speed.

Above all, $C(s) = T(s) + B(s)$ has a unique global minimum with pack size s^* , i.e., for all $s_1 < s_2 \leq s^*$ in S , $C(s_1) < C(s_2)$ holds, and for all $s^* \leq s_1 < s_2$ in S , $C(s_1) > C(s_2)$ holds. $\square$

Intuitively, if there is a pack size s , then any pack size $k * s$ will also include the packs of size s . Hence, there bit size cannot be smaller and Proposition 1 holds.

EXAMPLE 2. As shown in Figure 4, we present the storage cost $C(s)$, bit width cost $B(s)$ and value cost $T(s)$ of the pack sizes $[3, 6, \dots, 768]$, i.e., $\alpha\gamma^\beta$ with $\alpha = 3$ and $\gamma = 2$. $T(s)$ is increasing, and $B(s)$ is decreasing and convex. Consequently, their sum $C(s) = T(s) + B(s)$ is a function which has a unique global minimum (it first decreases and then increases).

3.1.2 Prune by Unimodality. Even when Proposition 1 (unimodality) holds, the algorithm does not only need to check prime numbers. In addition to primes, it is still necessary to examine composite pack sizes to identify where the encoding cost starts increasing, which enables effective pruning of subsequent candidates. We can reconsider $[1, 2, 3, 4, 5, 6, \dots]$ as two-dimensional arrays such as $[[1, 2, 4, 8, 16, 32, \dots], [3, 6, 12, 24, 48, \dots], [5, 10, 20, 40, \dots], \dots]$. In this scheme, once the cost of the pack size s in a given array begins to grow, the subsequent terms in that array can be disregarded, since

s_i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	...
$forward[]$	0	0	1	0	2	0	3	0	4	3	5	0	6	0	7	5	8	...
$increase[]$	F	F	F	F	T	F	T	F	T	F	F	F	T	F	F	F	T	...

Figure 5: Previous array for pruning with unimodality

the subsequent cost must be larger than $C(s)$ according to Proposition 1.

To apply Proposition 1, we could construct two arrays $forward[]$ and $increase[]$. $forward[]$ is an array of forward pack sizes and $increase[]$ is a Boolean array to indicate whether to begin to increase. The flag $increase[s]$ being True indicates that the storage cost for a pack size of s has become monotonically increasing; otherwise, the cost with s still decreases. These structures allow us to quickly check whether a given pack size has already been pruned by Proposition 1. Then, our algorithm uses $forward[]$ and $increase[]$ to prune pack size candidates in $[1, n]$, i.e., check the flag $increase[forward[s]]$ for each pack size s . If $increase[forward[s]]$ is True, we do not need to compute the storage cost of pack size s ; otherwise, calculate the cost $C(s)$ of s and check whether $C(s)$ is larger than $C(forward[s])$. If $C(s)$ is larger than $C(forward[s])$, the flag $increase[s]$ sets True.

EXAMPLE 3. As shown in Figure 5, if the storage cost $C(12)$ with pack size 12 is larger than $C(6)$, then $increased[12]$ is True. When considering the storage cost of pack size $s = 24$, we do not need to calculate cost of pack size $s = 12$ according to $increased[forward[24]] = increased[12] = True$.

3.2 Lower Bounds of the Cost Function

Motivated by the unimodality of the cost function $C(s)$ on a subset of pack sizes, we first investigate two lower bounds of $C(s)$, one increasing and the other decreasing, and use their intersection as the initial search pack size s . We can then prune candidates less than s and greater than s respectively, based on these bounds and the currently known best solution.

PROPOSITION 2 (NON-DECREASING LOWER BOUND). For a pack size s , $1 \leq s \leq n$, the non-decreasing lower bound $L(s)$ of the storage cost $C(s)$ is as follows,

$$C(s) \geq L(s) = s \max_{1 \leq i \leq s} y_i + \sum_{i=s+1}^n y_i + \lceil \log(\max_{1 \leq i \leq n} y_i + 1) \rceil.$$

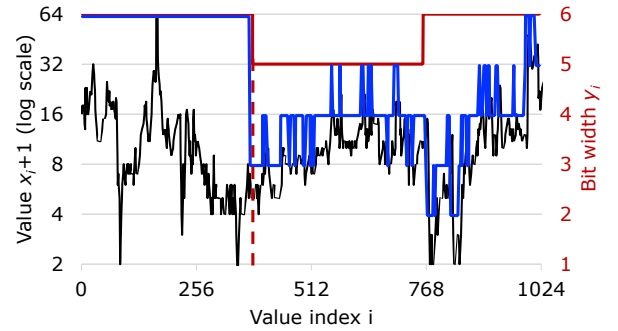
That is, for $1 \leq s_1 \leq s_2 \leq n$, the lower bound $L(s_1)$ must be no larger than $L(s_2)$,

$$L(s_1) \leq L(s_2).$$

Intuitively, the cost function for packing with pack size s will be no less than the cost of the first pack plus all subsequent values x_i using their own bit widths y_i . As s increases, the first pack becomes larger, and more values adopt larger bit widths, thus $L(s)$ is non-decreasing.

As shown in Figure 6(a), the blue line represents the lower bound $L(s)$ less than the cost $C(s)$ (the red line) when pack size s is 380. The lower bound $L(s)$ is increasing with s growing in Figure 6.

(a) Bit width y_i serves as lower bound $L(s)$ for $s = 380$
—X— $C(s)$ — $L(s)$



(b) Lower bound and cost under various s
— $C(s)$ — $L(s)$

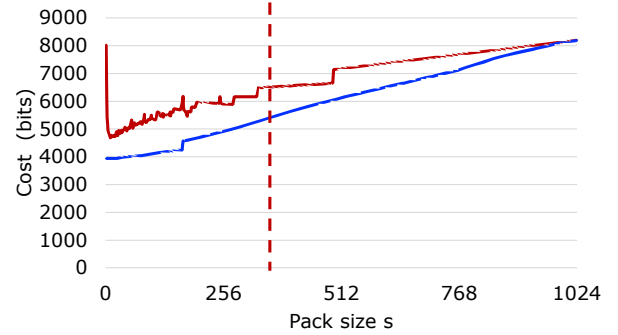


Figure 6: Non-decreasing lower bound $L(s)$ of $C(s)$

When the lower bound $L(s)$ exceeds the currently known best solution $C(s^*)$, we can prune any candidate pack size s that is greater than s^* .

In addition to a non-decreasing lower bound, we also derive a non-increasing upper bound as follows.

PROPOSITION 3 (NON-INCREASING LOWER BOUND). For a pack size s , $1 \leq s \leq n$, the non-increasing lower bound $L'(s)$ of the cost $C(s)$ is as follows,

$$C(s) \geq L'(s) = \sum_{i=1}^n y_i + \lceil \frac{n}{s} \rceil \lceil \log(\max_{1 \leq i \leq n} y_i + 1) \rceil.$$

That is, for $1 \leq s_1 \leq s_2 \leq n$, the lower bound $L'(s_1)$ must be no smaller than $L'(s_2)$,

$$L'(s_1) \geq L'(s_2).$$

Heuristically, the cost function with pack size s will be no less than the cost of all values x_i using their own bit widths y_i . As s grows, the number of packs decreases and the cost of bit widths decreases, thus $L'(s)$ is non-increasing.

As shown in Figure 7(a), the blue line represents the lower bound $L'(s)$ less than the cost $C(s)$ (the red line) when pack size s is 380. The lower bound $L'(s)$ is decreasing with s growing in Figure 7(b). When the lower bound $L'(s)$ exceeds the currently known best solution $C(s^*)$, we can prune any candidate pack size s that is smaller than s^* .

R1D3

R3W10 1

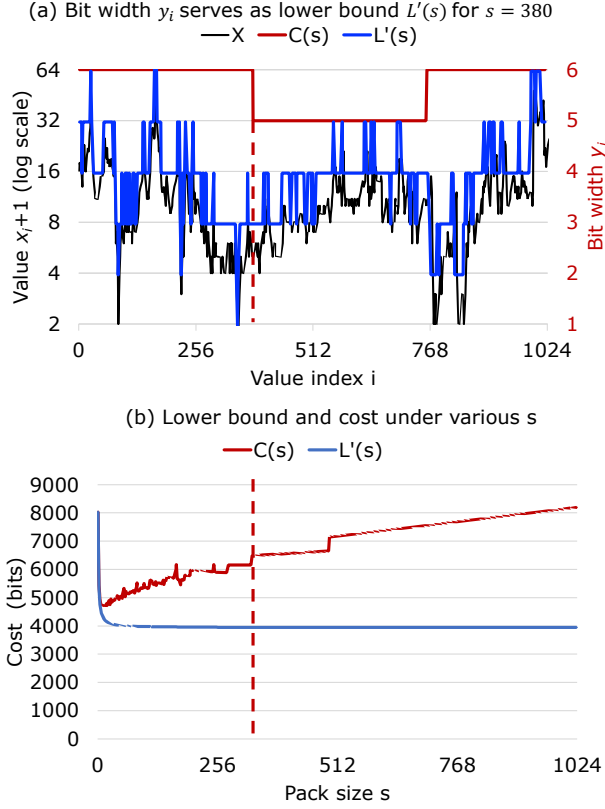


Figure 7: Non-increasing lower bound $L'(s)$ of $C(s)$

3.3 Monotonicity of the Cost Function

For the special case where the pack size s exceeds $\frac{n}{2}$ (resulting in only two packs), we formulate the following two propositions regarding the monotonicity of the cost function.

PROPOSITION 4 (NON-DECREASING COST). For $\frac{n}{2} \leq s \leq s_1 \leq s_2 < n$, if

$$\max_{s+1 \leq i \leq n-1} y_i \leq y_n \leq \max_{1 \leq i \leq s} y_i,$$

the cost $C(s)$ is not decreasing w.r.t. pack size s , i.e., the cost $C(s_1)$ must be no larger than $C(s_2)$,

$$C(s) \leq C(s_1) \leq C(s_2).$$

PROOF. Since $\frac{n}{2} \leq s \leq s_1 \leq s_2 < n$ holds, m is 2 and

$$C(s_1) = s_1 q_1 + (n - s_1) q_2 + 2 \lceil \log(\max_{1 \leq i \leq n} y_i + 1) \rceil,$$

$$C(s_2) = s_2 q'_1 + (n - s_2) q'_2 + 2 \lceil \log(\max_{1 \leq i \leq n} y_i + 1) \rceil.$$

Since $\frac{n}{2} \leq s \leq s_1 \leq s_2 < n$ satisfies, the following two conditions hold, $y_s \leq y_n$ and

$$q_1 = \max_{1 \leq i \leq s} y_i \geq \max_{s+1 \leq i \leq n} y_i = q_2,$$

$$q_2 = q'_2 = y_n \leq q_1 \leq q'_1.$$

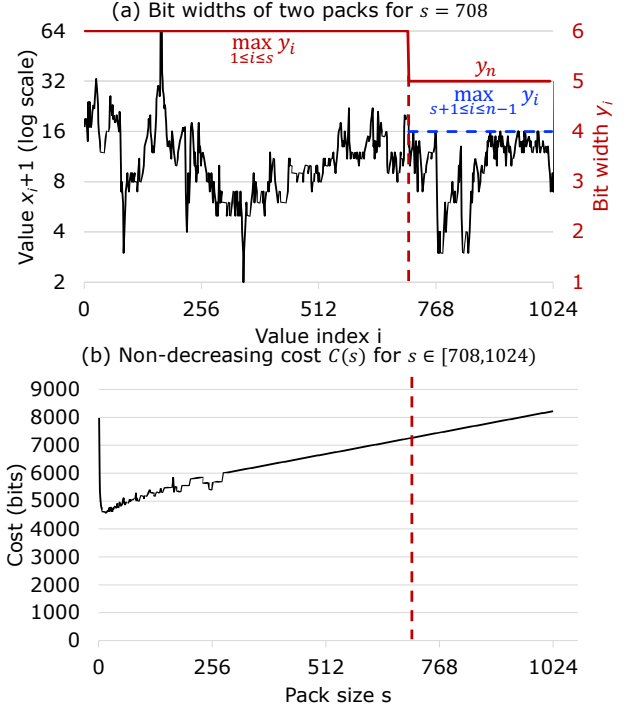


Figure 8: Example of non-decreasing cost $C(s)$ for $s \in [708, 1024)$ according to Proposition 4

Then, we could infer that

$$\begin{aligned} \Delta C(s) &= C(s_1) - C(s_2) \\ &= s_1 q_1 + (n - s_1) q_2 - (s_2 q'_1 + (n - s_2) q'_2) \\ &= s_1 q_1 - s_2 q'_1 + (s_2 - s_1) q_2 \\ &\leq s_1 q_1 - s_2 q_1 + (s_2 - s_1) q_2 \\ &\leq (s_1 - s_2)(q_1 - q_2) \leq 0. \end{aligned}$$

Therefore, $C(s_1) \leq C(s_2)$ holds and $C(s)$ is not decreasing. $\square$

Intuitively, if $\max_{s+1 \leq i \leq n-1} y_i \leq y_n \leq \max_{1 \leq i \leq s} y_i$ satisfies, the bit width q_2 of the second pack is smaller than that q_1 of the first pack. Thus, the more values have larger bit widths and the storage cost becomes larger with s increasing.

EXAMPLE 4. As shown in Figure 8(a), when the pack size i is larger than $s = 708$, $\max_{709 \leq i \leq 1023} y_i = 3 \leq y_n = 4 \leq \max_{1 \leq i \leq 708} y_i = 5$ holds. $C(s)$ is non-decreasing for $s \in [708, 1024)$, and the minimum cost $C(s)$ is $C(708)$ as shown in Figure 8(b). Thus, we do not need to compute the cost of pack size $s \in [708, 1024)$.

PROPOSITION 5 (NON-INCREASING COST). For $\frac{n}{2} \leq s \leq s_1 \leq s_2 < n$, if

$$\max_{s+1 \leq i \leq n-1} y_i \leq \max_{1 \leq i \leq s} y_i \leq y_n,$$

the cost $C(s)$ is not increasing w.r.t. pack size s , i.e., the cost $C(s_1)$ must be no smaller than $C(s_2)$,

$$C(s) \geq C(s_1) \geq C(s_2).$$

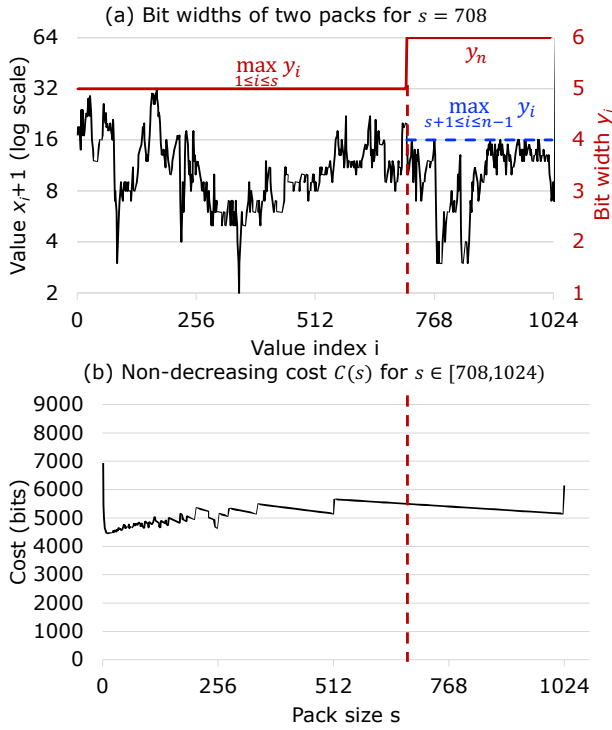


Figure 9: Example of non-increasing cost $C(s)$ for $s \in [708, 1024]$ according to Proposition 5

PROOF. Since $\max_{s+1 \leq i \leq n-1} y_i \leq \max_{1 \leq i \leq s} y_i \leq y_n$ satisfies, $q_1 = q'_1 < q_2 = q'_2 = y_n = y_{\max}$ holds true. We could infer that

$$\begin{aligned} \Delta C(s) &= C(s_1) - C(s_2) \\ &= s_1 q_1 + (n - s_1) q_2 - ((n - s_2) q'_1 + s_2 q'_2) \\ &= s_1 q_1 + (n - s_1) q_2 - s_2 q_1 - (n - s_2) q_2 \\ &= (s_1 - s_2) q_1 - (s_1 - s_2) q_2 \\ &= (s_1 - s_2) (q_1 - q_2) \geq 0. \end{aligned}$$

Therefore, $C(s_1) \geq C(s_2)$ holds and $C(s)$ is non-increasing. $\square$

Intuitively, when $\max_{s+1 \leq i \leq n-1} y_i \leq \max_{1 \leq i \leq s} y_i \leq y_n$ satisfies, the bit width q_2 of the second pack is larger than that q_1 of the first pack. Thus, with s growing, the more values have smaller bit widths and the storage cost becomes smaller.

EXAMPLE 5. As shown in Figure 9(a), when the pack size i is larger than $s = 708$, $\max_{709 \leq i \leq 1023} y_i = 3 \leq \max_{1 \leq i \leq 708} y_i = 4 \leq y_n = 5$ holds. $C(s)$ is non-increasing for $s \in [708, 1024]$, and the minimum cost $C(s)$ is $C(n - 1)$ as shown in Figure 9(b). Thus, we do not need to compute the cost of pack size $s \in [708, 1024]$.

3.4 Cost Function Computation

For different pack sizes s , many computations are repeated, which is time-consuming. Therefore, we construct a hash lookup table to facilitate rapid queries of pack bit-widths. Firstly, we construct a sparse table based on range maximum query (RMQ) $R[k][l]$ to

y_i	3	5	4	6	2	7	5	3	4	6	8	5	3	4	2	6
$R[1]$	5	5	6	6	7	7	5	4	6	8	8	5	4	4	6	
$R[2]$	6	6	7	7	7	7	6	8	8	8	5	5	6			
$R[3]$	7	7	7	7	8	8	8	8								
$R[4]$	8															

$\max\{y_1, \dots, y_{12}\} = \max\{R[3][1], R[3][5]\} = 8$

Figure 10: Sparse table constructed for range maximum query (RMQ)

store the maximum bit-width in any interval of length 2^k that begins at index l . The table is formally defined as $R[k][l] = \max_{l \leq i \leq l+2^k-1} y_i$. Given $l = js$ and $r = (j + 1)s$, we could get $q_j = \max_{l \leq i \leq r} \{y_i\} = \max\{R[k][l], R[k][r - 2^k + 1]\}$ with $k = \lceil \log(r - l + 1) \rceil$. For each pack p_j , we could use RMQ to obtain the maximum bit-width q_j within the corresponding interval. Then, compute the total cost $C(s)$ in Definition 1.

EXAMPLE 6. When the bit widths y_i are $[3, 5, 4, 6, 2, 7, 5, 3, 4, 6, 8, 5, 3, 4]$, we could construct the sparse table in Figure 10. For instance, we could get $R[1][1] = \max\{y_1, y_2\} = 5$. Then, when pack size s is 8, we could query q_1 with $\max\{R[3][1], R[3][5]\} = 8$.

Algorithm 1 presents the pseudo of the storage cost computation with RMQ. Firstly, compute m and initialize $C(s)$ in Line 1. Then, for each pack p_j , query the bit width q_j with RMQ in Lines 2-6. In Line 7, the algorithm computes the storage cost $C(s)$ of previous $m - 1$ packs according to Definition 1. Furthermore, the cost of the last pack and all bit width are accumulated into $C(s)$ in Lines 8-10. Finally, we compare the cost $C(s)$ with the optimal cost $C(s^*)$, and get the optimal cost $C(s^*)$ and s^* in Lines 11-13.

Algorithm 1: Cost Computation ($Cost(R, s, C(s^*), s^*)$)

Input: Sparse Table R , pack size s , optimal cost $C(s^*)$, optimal size s^*

Output: Cost $C(s)$, $C(s^*)$, s^*

```

1  $m \leftarrow \lceil \frac{n}{s} \rceil$ ,  $C(s) \leftarrow 0$ ;
2 for  $j \leftarrow 1$  to  $m - 1$  do
3    $l \leftarrow js$ ;
4    $r \leftarrow i_s + s - 1$ ;
5    $k \leftarrow \log(s)$ ;
6    $q_j \leftarrow \max(R[k][l], R[k][r - (1 \ll k) + 1])$ ;
7    $C(s) \leftarrow C(s) + sq_j$ ;
8  $l \leftarrow (m - 1)s$ ,  $r \leftarrow n$ ,  $k \leftarrow \log(r - l)$ ;
9  $q_m \leftarrow \max(R[k][l], R[k][r - (1 \ll k) + 1])$ ;
10  $C(s) \leftarrow C(s) + (n - (m - 1)s)q_m + m \lceil \log(\max_{1 \leq i \leq n} y_i + 1) \rceil$ ;
11 if  $C[s] < C(s^*)$  then
12    $C(s^*) \leftarrow C[s]$ ;
13    $s^* \leftarrow s$ ;
14 return  $C(s), C(s^*), s^*$ ;

```

show some notations on how a $T[k]$ computed, and how it is queried

range maximum query, or minimum, be consistent in the whole paper

may not emphasize the algorithm too much, existing work?

remove some proofs, if no space

what is the novelty of RMQ part

3.5 Determination Algorithm

In the section, we present the determination processing of optimal pack size s^* with pruning and RMQ in Algorithm 2. In Line 1, the algorithm begins by initializing several key variables and data structures. First, it sets the initial values for y_i , η , s' , θ , and s_s . Here, η indicates whether the condition $\max_{s+1 \leq i \leq n-1} y_i \leq y_n \leq \max_{1 \leq i \leq s} y_i$ (from Proposition 4) holds, and s' records the smallest s for which η becomes True. Similarly, θ tracks the condition $\max_{s+1 \leq i \leq n-1} y_i \leq \max_{1 \leq i \leq s} y_i \leq y_n$ from Proposition 5. The variable s_s marks the starting s where the inequality $L(s) < L'(s)$ begins to hold. Finally, the algorithm initializes the auxiliary arrays *forward*[1, ..., n] and *increased*[1, ..., n]. In Lines 2-3, the algorithm starts to compute the bit width y_i of the i -th values. We check whether conditions $\max_{s+1 \leq i \leq n-1} y_i \leq y_n \leq \max_{1 \leq i \leq s} y_i$ or conditions $\max_{s+1 \leq i \leq n-1} y_i \leq \max_{1 \leq i \leq s} y_i \leq y_n$ hold in Propositions 4 and 5. In Lines 8-10, calculate the lower bounds $L(s)$ and $L'(s)$ in Propositions 6 and 7, and determine the starting pack size s_s . And we calculate the sparse table for RMQ in Lines 11-14.

In Lines 16-19, we use $L'(s)$ to prune the pack size candidates. If the optimal cost $C(s^*)$ is smaller than $L'(s)$, the pack sizes smaller than s must not be the optimal one and do not need to iterate until 1. In Lines 20-24, we use $L(s)$ to prune the candidates according to Proposition 7. If the optimal cost $C(s^*)$ is smaller than $L(s)$, the pack sizes smaller than s must not be the optimal one and we do not need to iterate until s' . Otherwise, we prune the candidates of pack sizes with Proposition 1 in Lines 25-30. Finally, if $L(s)$ does not contribute to pruning, then Proposition 4 or 5 is applied for pruning in lines 31-35 of the algorithm.

Apache TsFile [32] is a high-performance columnar storage file format designed for industrial time-series data. Apache TsFile organizes data into chunk groups, and chunks consist of headers and pages, which are the basic serialization units. We implement Algorithm 2 in Apache TsFile.

4 Experiment

In this section, we evaluate compression performance of our methods. Our experimental setup is detailed in Section 4.1, covering the datasets, baseline algorithms, and metrics. In Section 4.2, we describe the motivation of our method with distribution of optimal pack size. Section 4.3 shows the improvement of bit-packing's compression ratio with optimal pack sizes. In Sections 4.5.1 and 4.5.2, we report the compression ratio, speed, and decompression time for our method and baselines under different pack and page sizes. And we evaluate the pruning rate of our methods on various datasets and varying page size in Section 4.6.

4.1 Experiment Setting

All experiments were conducted on a server equipped with two AMD EPYC 7713 CPUs (64 cores each), supporting the AVX2 instruction set, which is utilized by our SIMD implementation. The server is also equipped with five NVIDIA A100 GPUs (sm_80, Ampere architecture), which are used for the SIMT-based compression and decompression kernels.

4.1.1 Methods. We compare our pack size determination algorithm including three variants BP-All, BP-Prune and BP-Prune-RMQ, with

Algorithm 2: Optimal Pack Size Determination

Input: Array of values $X = (x_1, x_2, \dots, x_i, \dots, x_n)$
Output: Optimal pack size s^*

```

1  $y \leftarrow [0, \dots, 0]$ ,  $\eta = \text{False}$ ,  $s' = 0$ ,  $\theta = \text{False}$ ,
    $\text{increase}[1, \dots, n] \leftarrow [\text{False}, \dots, \text{False}]$ ,  $s_s = 16$ ;
2 for  $i \leftarrow n$  to 1 do
3    $y_i \leftarrow \lceil \log(x_i + 1) \rceil$ ;
4   if  $\max_{s+1 \leq i \leq n-1} y_i \leq y_n \leq \max_{1 \leq i \leq s} y_i$  then
5      $\eta = \text{True}$ ,  $s' = s$ ;
6   if  $\max_{s+1 \leq i \leq n-1} y_i \leq \max_{1 \leq i \leq s} y_i \leq y_n$  then
7      $\theta = \text{True}$ ;
8   Calculate  $L(s)$  and  $L'(s)$  in Propositions 6 and 7;
9   if  $L(s) < L'(s)$  and  $L(s+1) > L'(s+1)$  then
10     $s_s = s$ ;
11  $z \leftarrow \lceil \log(\max_{1 \leq i \leq n} y_i + 1) \rceil$ ,  $R[0][1 \dots n] \leftarrow y[1 \dots n]$ ;
12 for  $k \leftarrow 1$  to  $\lceil \log n \rceil - 1$  do
13   for  $i \leftarrow 1$  to  $n - 2^k + 1$  do
14      $R[k][i] \leftarrow \max(R[k-1][i], R[k-1][i + 2^{k-1}])$ ;
15  $s^* \leftarrow n$ ,  $C(s^*) \leftarrow nz + z$ ;
16 for  $s \leftarrow s_s$  to 1 do
17   if  $C(s^*) < L'(s)$  then
18     Break;
19    $C[s], C(s^*), s^* \leftarrow \text{Cost}(R, s, C(s^*), s^*)$ ;
20  $\text{isBreak} = \text{False}$ ;
21 for  $s \leftarrow s_s + 1$  to  $s'$  do
22   if  $C(s^*) < L(s+1)$  then
23      $\text{isBreak} = \text{True}$ ;
24     Break;
25   if  $\text{forward}[s] \neq 0$  and  $\text{increase}[\text{forward}[s]]$  then
26      $\text{increase}[s] \leftarrow \text{True}$ ;
27     Continue;
28    $C[s], C(s^*), s^* \leftarrow \text{Cost}(R, s, C(s^*), s^*)$ ;
29   if  $\text{forward}[s] \neq 0$  and  $C[s] > C[\text{forward}[s]]$  then
30      $\text{increase}[s] \leftarrow \text{True}$ ;
31 if not isBreak then
32   if  $\eta$  then
33      $C[s], C(s^*), s^* \leftarrow \text{Cost}(R, s', C(s^*), s^*)$ ;
34   if  $\theta$  then
35      $C[s], C(s^*), s^* \leftarrow \text{Cost}(R, n-1, C(s^*), s^*)$ ;
36 return  $s^*$ ;

```

bit-packing [33] and Sprintz [5]. BP-All computes the cost exhaustively without any pruning, BP-Prune applies Propositions 1-5 for pruning, and BP-Prune-RMQ further accelerates the computation by incorporating RMQ to reduce runtime. These three solutions are also integrated into Sprintz as replacements for its original bit-packing module, yielding the corresponding three methods: Sprintz-All, Sprintz-Prune, and Sprintz-Prune-RMQ.

4.1.2 Datasets. To verify performance, we utilize 17 real-world datasets from various domains, ensuring coverage of both time-series and non-time-series data. Table 2 presents the standard deviation (Stddev) and the minimum and maximum bit-widths (Min/Max) required to represent data points in each dataset. The sources and

Table 2: Real-world datasets

Dataset	Abbr.	Source	# Values	Stddev	Min/Max
Air-pressure	AP	[24]	137,721,453	4.55	1/24 bits
Bird-migration	BM	[12]	17,964	0.52	19/23 bits
Blockchain-tr	BTR	[8]	231,031	1.61	1/22 bits
Citylat	CLT	[19]	41,001	1.12	6/20 bits
Citylon	CLN	[19]	41,001	2.70	1/21 bits
City-temp	CT	[25]	2,905,887	1.07	9/13 bits
Cyber-Vehicle	CV		35,676,900	4.91	1/17 bits
Dewpoint-temp	DT	[23]	5,413,914	0.52	12/14 bits
Food-prices	FP	[27]	2,050,638	3.95	1/36 bits
IR-bio-temp	IR	[22]	380,817,839	2.51	1/11 bits
PM10-dust	PM10	[21]	222,911	4.45	1/18 bits
Stocks-DE	SDE	[7]	45,403,710	1.70	12/18 bits
Stocks-UK	SUK	[7]	115,146,731	1.67	14/20 bits
Stocks-USA	SUSA	[7]	374,428,996	1.03	11/15 bits
TY-Fuel	TF		183,556,352	2.36	1/7 bits
TY-Transport	TT		16,596,252	1.57	1/6 bits
Wind-speed	WS	[20]	199,570,396	1.03	1/8 bits

application scenarios of these datasets are detailed in the appendix [?].

Note that the commonly used floating-point datasets are often with fixed decimal precision. Following the same line of preprocessing by scaling [18], we map the floating-point numbers to integers, i.e., multiplying by 10^k where k is the number of significant digits after the decimal point. The complex structural representation of floating-point numbers (sign/exponent/mantissa) can thus be compressed more easily by bit-packing as simple integers.

4.1.3 Metrics. We evaluate the compression ratio measured as the ratio of uncompressed to compressed data size, i.e.,

$$\text{compressionRatio} = \frac{\text{uncompressedSize}}{\text{compressedSize}}.$$

Additionally, we measure compression and decompression time in CPU nanoseconds per point (ns/point). All results are averaged over 500 independent runs. For clarity, and unless stated otherwise, the results below (including Figures 14 and 15) present the average performance of each method across all datasets from Table 2.

4.2 Distribution of Optimal Pack Sizes

The optimal pack size is also very different when applied to different compression algorithms on the same values. The key reason is the trade-off between the bit-width cost and the value cost. As shown in Figure 3 and suggested by Proposition 1, the bit-width cost decreases as the pack size s increases, with a relatively steep reduction when s is small. In contrast, the value cost typically increases more slowly with s . However, after applying Sprintz, the local variability of values is amplified, making larger packs more likely to include outliers with higher bit-widths. This effect offsets the benefit of larger s , leading to smaller pack sizes being optimal in many cases, as observed in Figure 2(b).

Furthermore, We study the relationship between data smoothness and the relative compression improvement of our method comparing to fixed-pack-size method. We quantify smoothness as

the mean of difference between adjacent bit widths, i.e., mean of bit width delta. A larger mean of delta represents adjacent bit widths differ more on average, indicating that the data fluctuates more. As shown in Figure 11(a), we observe that datasets with larger fluctuations (i.e., less smooth) tend to benefit more from our method.

And we also analyze the impact of the bit-length range. By finding the optimal pack size, each value can save about 1 bit. However, this improvement becomes marginal when the bit length is large. As shown in Figure 11(b), a larger bit-length range generally correlates with smaller improvements.

Moreover, we explore whether the impact can be predicted using a learned model such as random forest model. Using the above features, the model already achieves strong predictive performance ($R^2 = 0.953$, $\rho = 0.977$) in Figure 11(c). By incorporating 8 additional features, including mean, standard deviation, median, change count, 25th percentile, 75th percentile, run-length mean of bit widths, and proportion of zeros, the prediction accuracy is further improved.

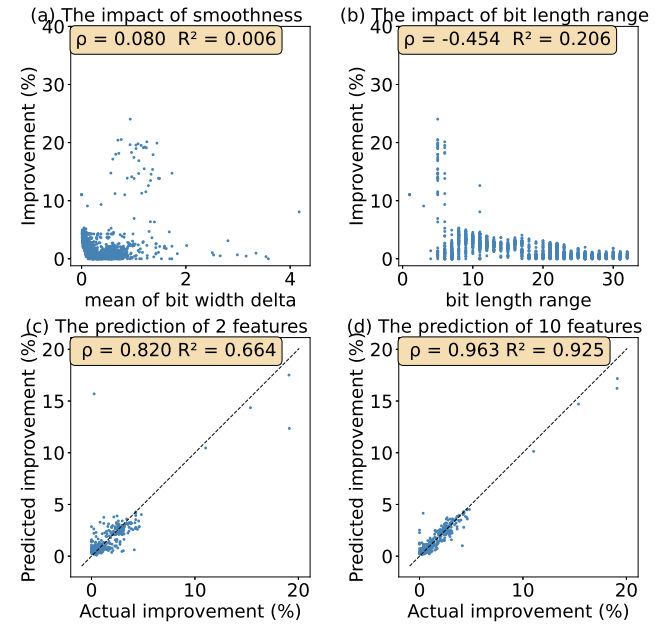


Figure 11: Data characters predict the impact of optimal pack sizes

4.3 Improvement of Optimal Pack Sizes

Figures 12(a) and (b) illustrate the percentage of improvement in compression ratio of Bit-packing and Sprintz achieved by selecting the optimal pack size for each individual page, compared to using a fixed pack size 16 for BP and 8 for Sprintz across all pages. The “improvement” is calculated as the compression ratio of BP (Prune-RMQ) or Sprintz (Prune-RMQ) minus that of BP or Sprintz, divided by that of BP or Sprintz.

Since the optimal pack size is not a fixed size (e.g., 8) for all pages, the method selecting the optimal pack size for each page has the improvement of compression ratio on many datasets in 12. Especially for data processed by Sprintz, the compression ratio shows

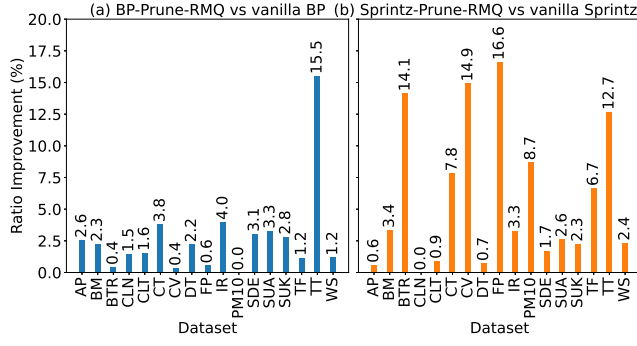


Figure 12: Improvement of compression ratio of BP-Prune-RMQ and Sprintz-Prune-RMQ

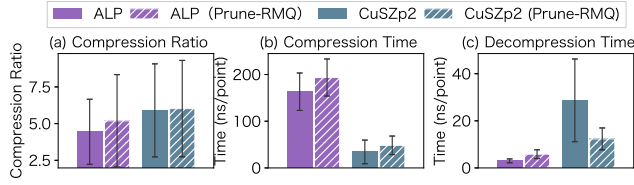


Figure 13: The compression performance of ALP and CuSZp2 with (without) optimal pack size

a more pronounced improvement, as many optimal pack sizes are smaller than 8, according to Figure 2. Note that this study does not propose a new compression method, but to determine a proper parameter of pack size for the vanilla Bit-packing encoding. It is thus not surprising that the improvement might not be significant, if the vanilla Bit-packing encoding already works well by setting a fixed pack size, as illustrated in Figure 12(a). However, selecting the optimal pack size for the vanilla Bit-packing encoding is costly, e.g., by testing all possible pack sizes. Our major contribution is thus to reduce the time cost of pack size determination, as presented in Figures 14.

Nevertheless, the 1 bit per value gain could be considerable, if the value bit-width is small. For example, for the new dataset TT, the difference between the maximum and minimum bit widths is 6 bits, saving 1 bit per value leads to a 15.5% improvement, in Figure 12(a). Moreover, bit-packing savings can accumulate. For example, our proposal can further improve Sprintz by optimizing the pack size, in Figure 12(b). The gain is generally more significant than that of Bit-packing, since the value width becomes smaller after Sprintz processing.

For variants of bit-packing, note that we do not propose a new compression algorithm, but optimize the parameter selection of the existing bit-packing encoding. Hence, our pack size determination can also be applied to ALP [3] and CuSZp2 [11], i.e., complementing them. As shown in Figure 13, the pack sizes determined by our pruning and RMQ techniques further improve the compression ratios of both ALP and CuSZp2. The improvement on CuSZp2 is not as significant as ALP, since it is lossless compression with high ratio already.

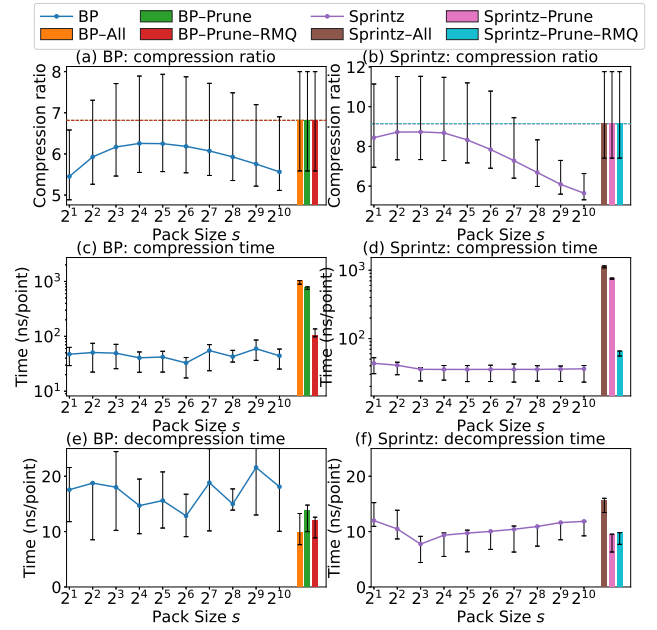


Figure 14: Performance under various fixed pack size s

4.4 Comparison with Related Work

Simple-8b uses a selector-based scheme that packs multiple integers into a 64-bit word with a uniform bit width. While efficiently decompressing, this design can be suboptimal if values within a group have different bit widths, leading to wasted space. In contrast, our method adapts the packing strategy with optimal pack size, resulting in better compression ratios. We also observe that our method achieves comparable compression and decompression time to Simple-8b in Figure 16.

Owing to the effective scaling preprocessing, the complex structural representation of floating-point numbers (sign/exponent/mantissa) can be compressed more easily by bit-packing as mapped integers. The corresponding compression ratio is thus better than Elf*.

4.5 Hyperparameter Sensitivity

4.5.1 Comparison with Various Fixed Pack Sizes. In this section, we evaluate the compression performance of Bit-Packing (BP) and Sprintz, including its compression ratio, compression time, and decompression time, under various fixed pack sizes s . The compression performance on variants of BP and Sprintz is also evaluated. According to Figures 14(a) and (b), both our pruning and RMQ optimizations successfully identify the optimal pack size, and all achieve higher compression ratios than the fixed-pack-size baseline. Moreover, Sprintz equipped with optimal pack size selection attains a better compression ratio than the corresponding BP-based method alone. Since many candidate pack sizes are pruned, the compression time of BP-Prune is significantly improved. Moreover, RMQ optimizes the cost calculation time for each remaining candidate pack size, leading to further time savings. As a result, the compression time of BP-Prune-RMQ is nearly as low as that of the baseline Bit-packing. The decompression time varies slightly with

R2D1 1

R2D2

R3W4
1(1)

R2D4 1

R3W2

R3W4
1(2)

this figure is confusing, should split in the future, (a)

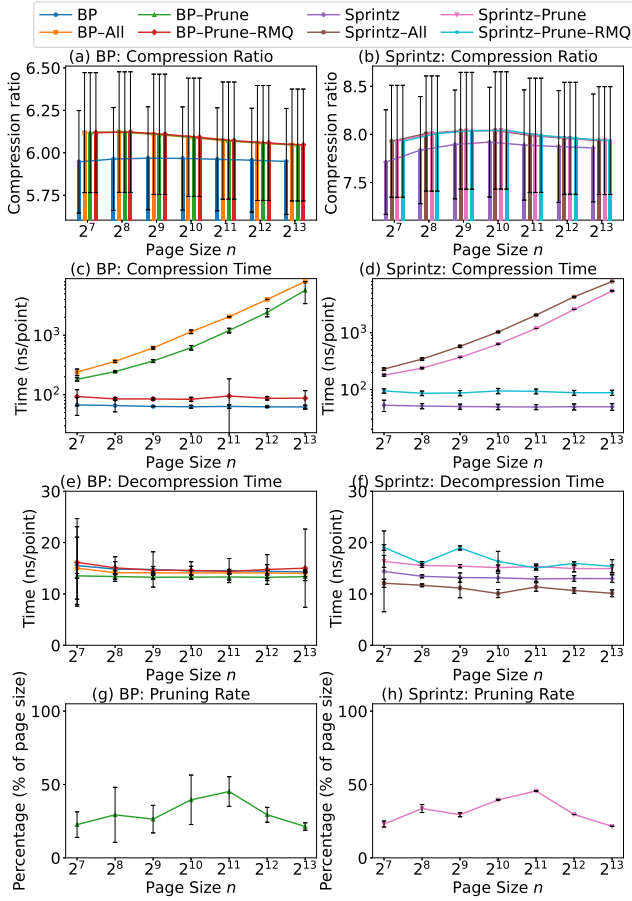


Figure 15: Performance under various page sizes n

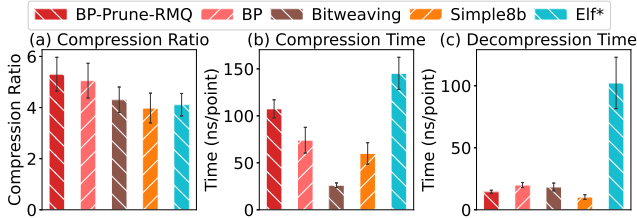


Figure 16: Comparison with Other Algorithms

different pack sizes, yet the decompression steps remain identical across all variants. Consequently, the decompression times of the different variants are nearly identical.

In the one-time compute, long-term storage scenario [9], e.g., in the cloud, data is compressed once but accessed many times throughout its lifecycle. This scenario justifies an optimization strategy with two key objectives: first, to improve the compression ratio, thereby reducing long-term cloud storage costs; and second, to ensure fast decompression speed for responsive data retrieval, even if a higher one-time compression overhead is incurred. Therefore, our proposed enhancement to bit-packing, i.e., BP-Prune-RMQ,

which significantly improves the compression ratio while keeping both compression and decompression times low, is well-suited to this practical requirement.

4.5.2 Evaluation under Various Page Sizes. We evaluate compression ratio, compression time, and decompression time of BP, BP-All, BP-Prune, and BP-Prune-RMQ varying page size n , and present the results in Figure 15. As page size begins to increase, BP-All, BP-Prune and BP-Prune-RMQ find better pack sizes, leading to an improved compression ratio. However, as n continues to grow, more pack sizes need to be considered, causing BP-All, BP-Prune and BP-Prune-RMQ to select pack sizes that are suboptimal for a portion of the values. Consequently, the compression ratio starts to degrade as shown in Figure 15(a). The same trend is observed for Sprintz and its variants in Figure 15(b). According to Figures 15(a) and (b), page size 2^8 is preferred for optimized BP, and 2^{10} for optimized Sprintz. Since BP-All and BP-Prune have $O(n^2)$ time complexity, as depicted in Figure 15, the average compression time per value for both methods increases at an accelerating rate with larger s . In contrast, BP-Prune-RMQ has $O(n \log n)$ complexity, thus its average compression time per value remains nearly constant as s increases.

4.6 Efficiency of Pruning

This study evaluates the efficiency variation of the pruning techniques proposed in Sections 3.1-3.3. [We define the pruning rate as the ratio of the number of pack size candidates pruned by our Propositions 1-5 to the total number of possible pack sizes \$n\$.](#)

In this section, we present experimental results on how the pruning rate varies with page size. The initial rise in the pruning rate is due to the heightened effectiveness of Proposition 1 as the page size expands, as depicted in Figures 15(g) and (h). However, a further increase in page size results in larger candidate pack sizes, which often lack smaller divisors. This structural property limits the applicability of Proposition 1, causing the pruning rate to decline. Thus, the gap in compression time between Bit-packing and BP-Prune exhibits a pattern of first increasing and then decreasing, as illustrated in Figures 15(c) and (d).

4.7 Comparing with Heuristic Methods

4.7.1 Comparing with Bit-packing with Sorting. Unlike time-series data, order might not be important for some non-time-series datasets. We test the compression performance with and without sorting for BP-All (i.e., traversing all possible pack sizes without pruning) and BP-Prune-RMQ (i.e., the method using pruning and RMQ to determine the pack size) on non-time-series datasets. If the values are sorted in descending order, values with similar bit width are together. As a result, sorting reduces the bit width needed for each value and makes them more packable. Hence, sorting improves the compression ratio of BP as shown in Figure 17(a). Moreover, the maximum bit width of the current pack can be determined by the first value, and thus the pack size can be estimated more quickly. The methods with sorting have less compression time in Figure 17(b).

4.7.2 Comparing with Bit-packing with Pre-defined Well-spaced Candidates. In this section, we evaluate a heuristic that selects from a

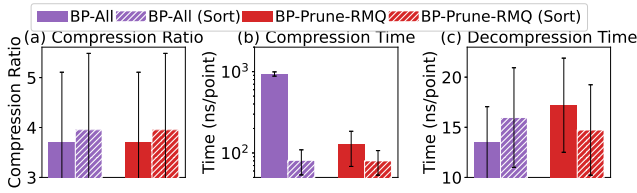


Figure 17: Impact of sorting non-time-series data on bit-packing with optimal pack size

set of pre-defined well-spaced candidate pack sizes (e.g., powers of two up to 2^{10}). The results are demonstrated in Figure 18.

The heuristic approach reduces compression time by approximately 5 nanosecond per point compared to our method in Figure 18, but leads to a slight degradation in compression ratio. This result highlights the trade-off between efficiency and optimality: while heuristic selection is faster, our method consistently achieves better compression ratio by identifying more precise pack sizes.

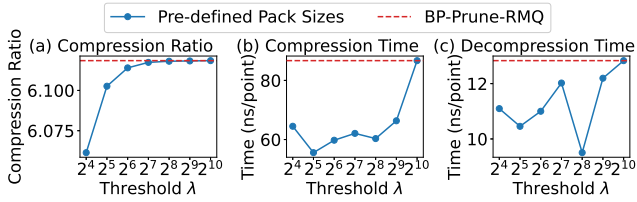


Figure 18: Compression performance of bit-packing with pre-defined pack sizes and all pack sizes

4.8 Optimizing Pack Size with SIMD

We present below how our proposal can work together with SIMD and SIMT on GPUs, and the corresponding trade-off evaluation in this section. Note that the data are compressed in pages in practice. We determine a fixed pack size for each page, and thus is compatible with vectorization. Moreover, pack sizes aligned with multiples of 8 are preferred, e.g., 16 in FastLanes [2] or SIMDComp [15], so that they can be easily batched in 256 bits for vectorization. When the selected optimal pack sizes do not aligned with multiples of 8, such as 15, vectorization might not be fully utilized. Hence, there is a trade-off on compression ratio and time.

Figure 19 shows that with vectorization, i.e., SIMD, FastLanes and SIMT, the compression time can be significantly improved compared with the scalar implementation. The optimal pack sizes determined by our pruning and RMQ techniques indeed improve the compression ratio with the extra cost on time, i.e., the trade-off. Nevertheless, for the SIMT implementation, encoding uses per-thread writes with precomputed bit positions and `atomicOr`, which is particularly effective for non-aligned pack sizes (not multiples of 8). By selecting the optimal pack size, since the compressed data size reduces, the compression time is even improved a bit.

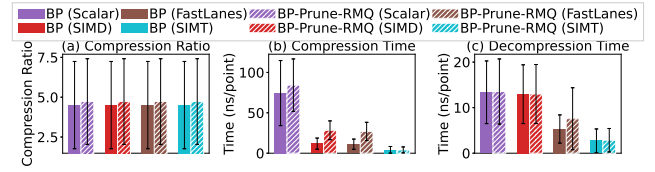


Figure 19: Compression performance of bit-packing optimal pack size with SIMDComp, Fastlane and SIMT

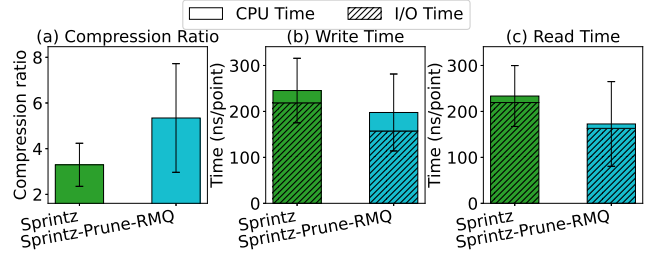


Figure 20: Compression performance impact after deployment in the real system

4.9 System Integration

We demonstrate the actual performance evaluation in the real system deployment of Apache TsFile in the section.

In addition to the CPU cost of compression and decompression, the system integration incurs IO cost in writing and reading the compressed data. In Figure 20(b), while determining the optimal pack size by our pruning and RMQ takes more CPU time, the corresponding IO cost is significantly reduced, due to better compression. The total write and read time costs of the system are indeed improved.

5 Related Work

Numerous data compression algorithms have been developed [6, 28, 31]. Among these, lossless compression is critical in domains such as the industrial Internet of Things and financial record-keeping, as it guarantees exact data reconstruction. Recent studies [28–30] have introduced various lossless methods with improved efficiency and compression performance. However, many techniques incur significant time overhead [1, 13], motivating a focus on lightweight lossless approaches, such as bit-packing [33] and Sprintz [5].

Directly, operating on the raw floating-point representation (sign/exponent/mantissa) is often inefficient, since even small fluctuations in values can lead to large differences in their binary representation. We first convert floating-point values into integers via scaling (i.e., multiplying by a fixed factor based on the maximum decimal precision), and then apply bit-packing to the integers.

Simple-8b [4] uses a selector-based scheme that packs multiple integers into a 64-bit word with a uniform bit width. While efficient, this design can be suboptimal when values within a group have different bit widths, leading to wasted space. Bit-packing (BP) is widely used as an effective operator for data compression [5, 14, 33]. Bit-packing groups all values in a pack using a uniform bit

width, which is determined by the bit width of the pack's maximum value after leading zero bits are removed. BitWeaving [16] also focuses on bit-level data layouts to accelerate scan performance, without targeting improvements in the compression ratio of bit-packed representations. **FastLanes [2] primarily addresses the layout and pipelining of bit-packed data to maximize decompression throughput, rather than improving the compression ratio of bit-packing itself.**

Sprintz reduces the magnitude of the values through delta encoding, thereby lowering the bit width required for storage and consequently decreasing the overall storage space [5]. CodecDB [13] evaluates delta-encoding combined with bit-packing as a lightweight encoding option within an encoding-selection framework. However, most algorithms with bit-packing do not improve the compression ratio by selecting the optimal pack size for values.

6 Conclusion

This study focuses on the pack size optimization problem for bit-packing. We propose an efficient solution based on candidate pack size pruning and range maximum query, which improve the compression ratio of bit-packing without much extra time cost. The pruning strategy reduces the candidates of pack sizes by leveraging unimodality, lower bounds, and monotonicity properties, while range maximum query uses a sparse table to efficiently calculate the bit width of a pack. Our proposal can be applied to improve any compression methods that use bit-packing, such as Sprintz. We deploy our algorithms in Apache TsFile and evaluate them on real-world datasets. The experimental results demonstrate that the pack size optimization indeed significantly improves the compression ratio of bit-packing with no more 20% compression time. This trade-off is highly worthwhile, especially for the *one-time compute long-term storage* scenario in the cloud.

Acknowledgments

This work is supported in part by National Key Research and Development Plan (2025ZD1601701, 2024YFB3311901, 2024YFB3310200, 2021YFB3300500), Natural Science Foundation of China (62232005, 92267203, 62021002), the State Grid Ningxia Electric Power Co. Science and Technology Project (SGNXYX00SCJS2400058), Beijing National Research Center for Information Science and Technology (BNR2025RC01011), and Beijing Key Laboratory of Industrial Big Data System and Application. Shaoxu Song (<https://sxsong.github.io/>) is the corresponding author.

References

- [1] Daniel J. Abadi, Samuel Madden, and Miguel Ferreira. Integrating compression and execution in column-oriented database systems. In Surajit Chaudhuri, Vagelis Hristidis, and Neoklis Polyzotis, editors, *Proceedings of the ACM SIGMOD International Conference on Management of Data, Chicago, Illinois, USA, June 27-29, 2006*, pages 671–682. ACM, 2006.
- [2] Azim Afrozeh, Leonardo X. Kuffo, and Peter A. Boncz. The fastlanes compression layout: Decoding >100 billion integers per second with scalar code. *Proc. VLDB Endow.*, 16(9):2132–2144, 2023.
- [3] Azim Afrozeh, Leonardo X. Kuffo, and Peter A. Boncz. ALP: adaptive lossless floating-point compression. *Proc. ACM Manag. Data*, 1(4):230:1–230:26, 2023.
- [4] Vo Ngoc Anh and Alistair Moffat. Index compression using 64-bit words. *Softw. Pract. Exp.*, 40(2):131–147, 2010.
- [5] Davis W. Blalock, Samuel Madden, and John V. Guttag. Sprintz: Time series compression for the internet of things. *Proc. ACM Interact. Mob. Wearable Ubiquitous Technol.*, 2(3):93:1–93:23, 2018.
- [6] Giacomo Chiarot and Claudio Silvestri. Time series compression survey. *ACM Comput. Surv.*, 55(10):198:1–198:32, 2023.
- [7] Financial dataset used in INFORE project. https://zenodo.org/records/3886895#Y4DdzHZByM_, 2023.
- [8] Blockchair Database Dumps. <https://gz.blockchair.com/bitcoin/transactions/>, 2023.
- [9] Dominik Durner, Viktor Leis, and Thomas Neumann. Exploiting cloud object storage for high-performance analytics. *Proc. VLDB Endow.*, 16(11):2769–2782, 2023.
- [10] Jonathan Goldstein, Raghu Ramakrishnan, and Uri Shaft. Compressing relations and indexes. In Susan Darling Urban and Elisa Bertino, editors, *Proceedings of the Fourteenth International Conference on Data Engineering, Orlando, Florida, USA, February 23-27, 1998*, pages 370–379. IEEE Computer Society, 1998.
- [11] Yafan Huang, Sheng Di, Guanpeng Li, and Franck Cappello. cuszp2: A GPU lossy compressor with extreme throughput and optimized compression ratio. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis, SC 2024, Atlanta, GA, USA, November 17-22, 2024*. IEEE, 2024.
- [12] InfluxDB. Influxdb 2.0 sample data. <https://github.com/influxdata/influxdb2-sample-data>, 2023.
- [13] Hao Jiang, Chunwei Liu, John Paparrizos, Andrew A. Chien, Jihong Ma, and Aaron J. Elmore. Good to the last bit: Data-driven encoding with codecdb. In Guoliang Li, Zhanhuai Li, Stratos Idreos, and Divesh Srivastava, editors, *SIGMOD '21: International Conference on Management of Data, Virtual Event, China, June 20-25, 2021*, pages 843–856. ACM, 2021.
- [14] Daniel Lemire and Leonid Boytsov. Decoding billions of integers per second through vectorization. *Softw. Pract. Exp.*, 45(1):1–29, 2015.
- [15] Lemire, Daniel and Boytsov, Leonid. Simd compression and the intersection of sorted integers, 2012.
- [16] Yinan Li and Jignesh M. Patel. Bitweaving: fast scans for main memory data processing. In Kenneth A. Ross, Divesh Srivastava, and Dimitris Papadias, editors, *Proceedings of the ACM SIGMOD International Conference on Management of Data, SIGMOD 2013, New York, NY, USA, June 22-27, 2013*, pages 289–300. ACM, 2013.
- [17] Zheng Li, Ruiyuan Li, Xiaolong Xu, Yi Wu, Chao Chen, Tong Liu, Jiaxing Shang, and Yu Zheng. Adaptive encoding strategies for lossless floating-point compression. *IEEE Internet Things J.*, 12(14):26071–26085, 2025.
- [18] Chunwei Liu, Hao Jiang, John Paparrizos, and Aaron J. Elmore. Decomposed bounded floats for fast compression and queries. *Proc. VLDB Endow.*, 14(11):2586–2598, 2021.
- [19] Kuntal Maity. World cities of different countries, 2021.
- [20] National Ecological Observatory Network (NEON). 2d wind speed and direction (dp1.00001.001). <https://data.neonscience.org/data-products/DP1.00001.001/RELEASE-2022>, 2022.
- [21] National Ecological Observatory Network (NEON). Dust and particulate size distribution (dp1.00017.001). <https://data.neonscience.org/data-products/DP1.00017.001/RELEASE-2022>, 2022.
- [22] National Ecological Observatory Network (NEON). Ir biological temperature (dp1.00005.001). <https://data.neonscience.org/data-products/DP1.00005.001/RELEASE-2022>, 2022.
- [23] National Ecological Observatory Network (NEON). Relative humidity above water on-buoy (dp1.20271.001). <https://data.neonscience.org/data-products/DP1.20271.001/RELEASE-2022>, 2022.
- [24] NEON (National Ecological Observatory Network). Barometric pressure, 2022. Data product DP1.00004.001, Release 2022.
- [25] Daily Temperature of Major Cities. <https://www.kaggle.com/sudalairajkumar/daily-temperature-of-major-cities>, 2023.
- [26] Apache TsFile. <https://github.com/apache/tsfile/tree/research/encoding-pack-size>, 2025.
- [27] World Food Programme (WFP). Global food prices database, 2023.
- [28] Tianrui Xia, Jinzhao Xiao, Yuxiang Huang, Changyu Hu, Shaoxu Song, Xiangdong Huang, and Jian-min Wang. Time series data encoding in apache iotdb: comparative analysis and recommendation. *VLDB J.*, 33(3):727–752, 2024.
- [29] Jinzhao Xiao, Wendi He, Shaoxu Song, Xiangdong Huang, Chen Wang, and Jianmin Wang. REGER: reordering time series data for regression encoding. In *40th IEEE International Conference on Data Engineering, ICDE 2024, Utrecht, The Netherlands, May 13-16, 2024*, pages 1242–1254. IEEE, 2024.
- [30] Jinzhao Xiao, Yuxiang Huang, Changyu Hu, Shaoxu Song, Xiangdong Huang, and Jianmin Wang. Time series data encoding for efficient storage: A comparative analysis in apache iotdb. *Proc. VLDB Endow.*, 15(10):2148–2160, 2022.
- [31] Jiujiang Zhang, Zhitao Shen, Shiyu Yang, Linghai Meng, Chuan Xiao, Wei Jia, Yue Li, Qinhui Sun, Wenjie Zhang, and Xuemin Lin. High-ratio compression for machine-generated data. *Proc. ACM Manag. Data*, 1(4):245:1–245:27, 2023.
- [32] Xin Zhao, Jialin Qiao, Xiangdong Huang, Chen Wang, Shaoxu Song, and Jianmin Wang. Apache tsfile: An iot-native time series file format. *Proc. VLDB Endow.*, 17(12):4064–4076, 2024.
- [33] Marcin Zukowski, Sándor Héman, Niels Nes, and Peter A. Boncz. Super-scalar RAM-CPU cache compression. In Ling Liu, Andreas Reuter, Kyu-Young Whang,

and Jianjun Zhang, editors, *Proceedings of the 22nd International Conference on Data Engineering, ICDE 2006, 3-8 April 2006, Atlanta, GA, USA*, page 59. IEEE Computer Society, 2006.

A Lower Bounds of the Cost Function

PROPOSITION 6 (NON-DECREASING LOWER BOUND). *For a pack size $s, 1 \leq s \leq n$, the non-decreasing lower bound $L(s)$ of the storage cost $C(s)$ is as follows,*

$$C(s) \geq L(s) = s \max_{1 \leq i \leq s} y_i + \sum_{i=s+1}^n y_i + \lceil \log(\max_{1 \leq i \leq n} y_i + 1) \rceil.$$

That is, for $1 \leq s_1 \leq s_2 \leq n$, the lower bound $L(s_1)$ must be no larger than $L(s_2)$,

$$L(s_1) \leq L(s_2).$$

PROOF. According to Definition 1, we could get that

$$\begin{aligned} C(s) &= s \sum_{j=1}^{m-1} q_j + (n - (m-1)s)q_m + m \lceil \log(\max_{1 \leq i \leq n} y_i + 1) \rceil \\ &\geq sq_1 + s \sum_{j=2}^{m-1} q_j + (n - (m-1)s)q_m + \lceil \log(\max_{1 \leq i \leq n} y_i + 1) \rceil \\ &\geq sq_1 + \sum_{i=s+1}^{s(m-1)} y_i + \sum_{i=s(m-1)+1}^n y_i + \lceil \log(\max_{1 \leq i \leq n} y_i + 1) \rceil \\ &\geq s \max_{1 \leq i \leq s} y_i + \sum_{i=s+1}^n y_i + \lceil \log(\max_{1 \leq i \leq n} y_i + 1) \rceil = L(s). \end{aligned}$$

For $1 \leq s_1 \leq s_2 \leq n$, we have $\max_{1 \leq i \leq s_1} y_i \leq \max_{1 \leq i \leq s_2} y_i$, and the difference $\Delta L(s)$ between the lower bound $L(s_1)$ and $L(s_2)$ is as follows,

$$\begin{aligned} \Delta L(s) &= L(s_1) - L(s_2) \\ &= s_1 \max_{1 \leq i \leq s_1} y_i + \sum_{i=s_1+1}^n y_i - (s_2 \max_{1 \leq i \leq s_2} y_i + \sum_{i=s_2+1}^n y_i) \\ &= s_1 \max_{1 \leq i \leq s_1} y_i - s_2 \max_{1 \leq i \leq s_2} y_i + \sum_{i=s_1+1}^{s_2} y_i \\ &= s_1 (\max_{1 \leq i \leq s_1} y_i - \max_{1 \leq i \leq s_2} y_i) + (s_1 - s_2) \max_{1 \leq i \leq s_2} y_i + \sum_{i=s_1+1}^{s_2} y_i \\ &\leq s_1 (\max_{1 \leq i \leq s_2} y_i - \max_{1 \leq i \leq s_2} y_i) \\ &\quad + (s_1 - s_2) \max_{1 \leq i \leq s_2} y_i + (s_2 - s_1) \max_{1 \leq i \leq s_2} y_i = 0. \end{aligned}$$

Therefore, we could conclude $L(s_1) \leq L(s_2)$. $\square$

PROPOSITION 7 (NON-INCREASING LOWER BOUND). *For a pack size $s, 1 \leq s \leq n$, the non-increasing lower bound $L'(s)$ of the cost $C(s)$ is as follows,*

$$C(s) \geq L'(s) = \sum_{i=1}^n y_i + \lceil \frac{n}{s} \rceil \lceil \log(\max_{1 \leq i \leq n} y_i + 1) \rceil.$$

That is, for $1 \leq s_1 \leq s_2 \leq n$, the lower bound $L'(s_1)$ must be no smaller than $L'(s_2)$,

$$L'(s_1) \geq L'(s_2).$$

PROOF. According to Definition 1, we could get that

$$\begin{aligned} C(s) &= s \sum_{j=1}^{m-1} q_j + (n - (m-1)s)q_m + m \lceil \log(\max_{1 \leq i \leq n} y_i + 1) \rceil \\ &\geq \sum_{i=1}^{s(m-1)} y_i + \sum_{i=s(m-1)+1}^n y_i + \lceil \frac{n}{s} \rceil \lceil \log(\max_{1 \leq i \leq n} y_i + 1) \rceil \\ &\geq \sum_{i=1}^n y_i + \lceil \frac{n}{s} \rceil \lceil \log(\max_{1 \leq i \leq n} y_i + 1) \rceil = L'(s). \end{aligned}$$

For $1 \leq s_1 \leq s_2 \leq n$, we have $\lceil \frac{n}{s_1} \rceil \geq \lceil \frac{n}{s_2} \rceil$, and the difference $\Delta L'(s)$ between the lower bound $L'(s_1)$ and $L'(s_2)$ is as follows,

$$\begin{aligned} \Delta L'(s) &= L'(s_1) - L'(s_2) \\ &= \sum_{i=1}^n y_i + \lceil \frac{n}{s_1} \rceil \lceil \log(\max_{1 \leq i \leq n} y_i + 1) \rceil \\ &\quad - (\sum_{i=1}^n y_i + \lceil \frac{n}{s_2} \rceil \lceil \log(\max_{1 \leq i \leq n} y_i + 1) \rceil) \\ &= (\lceil \frac{n}{s_1} \rceil - \lceil \frac{n}{s_2} \rceil) \lceil \log(\max_{1 \leq i \leq n} y_i + 1) \rceil \geq 0. \end{aligned}$$

Therefore, we could conclude $L'(s_1) \geq L'(s_2)$. $\square$

B Detailed Description of Datasets

The NEON datasets, sourced from the National Science Foundation's National Ecological Observatory Network, consist of five distinct time-series collections: Air-pressure (AP), Dew point-temp (DT), IR-bio-temp (IR), PM10-dust (PM10), and Wind-speed (WS). These datasets provide near real-time environmental measurements, capturing barometric pressure fluctuations, relative dew point temperatures over aquatic bodies, temperature variations in infrared-sensitive organisms, concentrations of PM10 particulate matter, and wind speed metrics, respectively. The City-temp (CT) dataset archives historical and contemporary temperature records from major urban centers across the globe. Financial data is represented by three stock market indices: Stocks-DE (SDE), Stocks-UK (SUK), and Stocks-USA (SUSA), which contain historical exchange price data for Germany, the United Kingdom, and the United States, respectively. From the InfluxDB repository, we include several specialized datasets: Bird-migration (BM), which tracks avian movement trajectories and positional data; and Blockchain-tr (BTR), which records daily aggregate values of Bitcoin transactions. Geospatial information is encapsulated in the Citylat (CLT) and Citylon (CLN) datasets, containing the latitude and longitude coordinates of cities worldwide. The Food-prices (FP) dataset, curated by the World Food Programme, documents global commodity price records for food items. Cyber-Vehicle (CV) tracks series data pertaining to the operational status of concrete mixer trucks. TY-Fuel (TF) comprises data concerning the fuel consumption of various vehicle engines. TY-Transport (TT) encompasses data associated with the transportation duration of trucks.

C Pruning Rate on Various Datasets

Figure 21 presents pruning rate of our methods with Propositions 1-5 on various datasets. Since Propositions 5 and 4 prunes close to half of the data in most cases, all datasets allow pruning of over

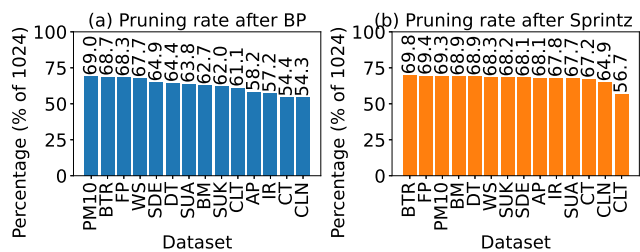


Figure 21: Pruning rate of candidate pack sizes on various datasets

half the candidate pack sizes in Figure 21. Thus, BP-Prune reduces the compression time by more than one third compared to BP-All in Figure 14(c). After processing with Sprintz, the value exhibits greater variability, which more frequently satisfies the preconditions of propositions, thereby enabling even more aggressive pruning than bit-packing. As a result, Sprintz-Prune halves the compression time compared to Sprintz-All, as shown in Figure 14(d).